

Shibboleth SSO 検証環境 構築手順書（純 Windows 版）

第 0.13 版

2026-07-10

0. 改訂履歴

版	日付	変更概要	備考
0.1	2026-07-05	純 Windows 版として新規作成。\$1～\$4（目的・前提・全体アーキテクチャ・パラメータ・ロードマップ）を記載	WSL 版 v0.15 を土台に、顧客 PLM 検証環境の再現用として起こす
0.2	2026-07-05	フェーズ 1（検証環境の初期設定：前提確認・時刻同期・着手前チェックポイント）を追記。推奨アカウント値（わかりやすさ優先）を \$3 に追記	入れ子仮想化は不要のため簡素化
0.3	2026-07-05	フェーズ 2（hosts・内部 CA・idp/sp サーバ証明書(a)の発行・PFX 化・信頼登録）を追記。証明書作成は Git for Windows 同梱の openssl を使用	ネットワーク方式選定は不要（同一 Windows・127.0.0.1）
0.4	2026-07-05	フェーズ 3（ApacheDS 導入・パーティション作成・ou=people・テストユーザー 01PLM01/02・検索バインド idp-reader）を追記	LDAP は ApacheDS。GUI（Directory Studio）+ LDIF
0.5	2026-07-08	フェーズ 3 の実機知見を反映（Java 前倒し／zip 優先方針／C:統一／ApacheDS は LDAP ポート 10389・設定は ou=config 方式で config.ldif 無し→既定パーティション dc=example,dc=com 流用／Directory Studio は exe 起動／userPassword は SSHA 自動ハッシュ）。\$6.4.1「証明書コマンドの解説」を追記	実機検証の反映と解説の追加
0.6	2026-07-08	フェーズ 4（OpenJDK は \$5.6 で導入済み。Tomcat 10.1 の zip 展開・環境変数・service.bat によるサービス化・8080 起動確認）を追記	Tomcat は zip + service.bat
0.7	2026-07-08	フェーズ 5（Shibboleth IdP 5：zip 展開 + install.bat 導入・context フラグメント配置・ldap.properties（10389/dc=example,dc=com/idp-reader）・emailAddress NameID の元 mail・起動確認）を追記。フェーズ 4 のサービス	本構成の山場

版	日付	変更概要	備考
0.8	2026-07-09	ス表示名を実機値「Apache Tomcat 10.1 Tomcat10」に補足 フェーズ 5 実機反映：§9.3 を訂正し JSTL (API+Glassfish 実装の 2 jar) 追加+build.bat 再ビルドを必須手順 に (未追加だと /idp/profile/status が ClassNotFoundException: jakarta.servlet.jsp.jstl.core.ConfigServletException) 。 §9.4 に trustCertificates/trustStore のコメントアウト、saml-nameid の bean 重複回避を追記	JSTL は必須だった
0.9	2026-07-09	フェーズ 6 (Tomcat 直 HTTPS 8443 公開：idp.pfx を conf に配置し server.xml に SSLHostConfig+Certificate の 8443 コネクタを追加、8080 は localhost 限定、鍵マーク確認) を追記。 Apache 前段は不要	WSL 版の Apache+mirrored を Tomcat コネクタ 1 つで代替
0.10	2026-07-09	フェーズ 7 (IIS 導入・sp.pfx 取込・既定サイトに 443/HTTPS バインド・確認用 whoami.asp) を追記。 sp.pfx は C:\lab\ca から取込。 UAC 承認モード差異・ASP 文字コードの補足を反映	WSL 版フェーズ 7 を流用 (mirrored 不要)
0.11	2026-07-09	フェーズ 8 (Shibboleth SP：MSI 導入・IIS ネイティブモジュール確認・shibboleth2.xml 編集 (ISAPI/RequestMapper/entityID/SSO) ・keygen で鍵(b)・サイト全体保護) を追記	WSL 版フェーズ 8 を流用
0.12	2026-07-09	フェーズ 8 実機反映 (MSI 表記「Configure IIS support」／サービス表示名「Shibboleth Daemon (Default)」／shibd.exe は sbin64 または sbin／使用インストーラ版数一覧)。 フェーズ 9 (メタデータ相互登録・初回 SSO・:8443 補正は不要) を追記	:8443 補正が不要になり簡素化
0.13	2026-07-10	フェーズ 9 実機反映：§13.1 を訂正し :8443 補正は必要 (install.bat 生成のメタデータはエンドポイントがポートなしのため) に。 フェーズ 10 (emailAddress 形式 NameID を mail から生成し REMOTE_USER に載せる：目標 01PLM01@plm-lab.local) を追記	uid/unspecified-mail/emailAddress

本書は、WSL 版構築手順書 (WSL2 上に IdP スタックを構築した学習環境) を土台に、**WSL を一切使わない純 Windows 構成**で顧客 PLM 検証環境を再現するための手順書です。SP

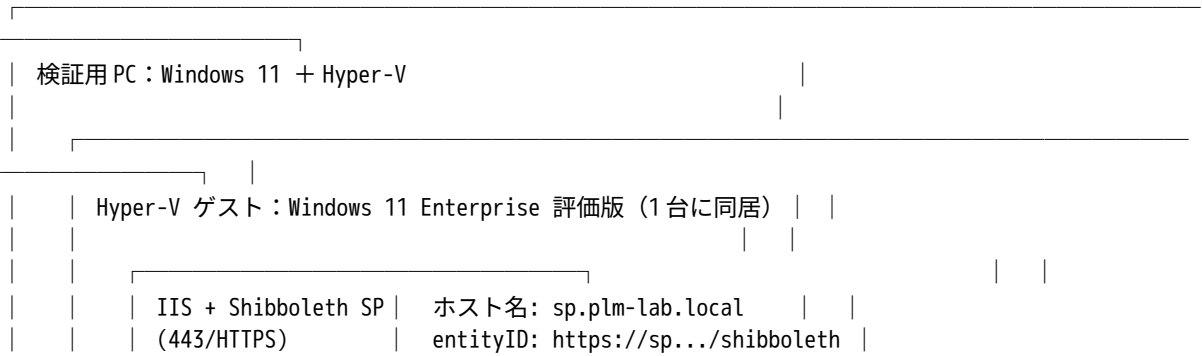
側（IIS+Shibboleth SP）と SAML 設計の考え方は WSL 版から流用し、IdP 側（Tomcat/Shibboleth IdP）・LDAP を Windows ネイティブに置き換えています。

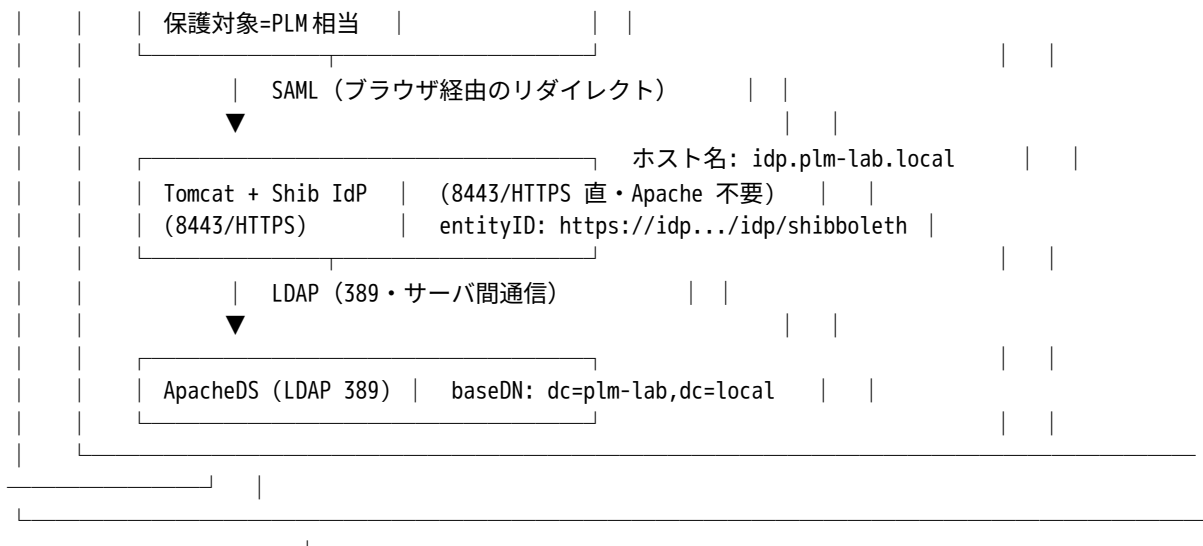
1. 目的と前提

- ・ **目的**：顧客 PLM 環境（Shibboleth=SP、前段=IIS、PLM は C:\inetpub\wwwroot 配下の IIS アプリ、IdP=Entra ID／独自認証）に近い SAML SSO 検証環境を、**全メンバーが習熟している Windows のみ**で構築し、SP 側の設定と動作を確認する。
- ・ **方針**：コンテナや WSL を使わず、**すべて Windows 上**で手作業構築する。IdP は学習用に自前構築し、将来 Entra ID に差し替え可能な形にする。
- ・ **物理ホスト**：検証用 PC（Windows 11、Hyper-V 有効）。
- ・ **検証用 Windows**：Hyper-V 仮想マシン上の **Windows 11 Enterprise 評価版**（90 日／slmgr /rearm で延長。付録 A）。
- ・ **役割分担（すべて同一のゲスト Windows 上に同居）**：
 - **IIS + Shibboleth SP**：ブラウザからの入口（前段）。未認証を IdP へ、認証後は REMOTE_USER を確定し、wwwroot 配下の PLM 相当アプリを実行。
 - **Tomcat + Shibboleth IdP**：学習用テスト IdP（HTTPS 8443 を直接公開）。将来 Entra ID に差し替え可能。
 - **ApacheDS（LDAP）**：ユーザーディレクトリ。IdP が認証・属性取得のために参照。
- ・ **前提（電源管理）**：物理ホスト・ゲスト仮想マシンとも **Windows のスリープ／休止は無効化**していること（組織の対象 PC も同様の運用）。本書はこの前提に立ち、スリープ復帰に伴う時刻ずれ対策は主手順から除外する（参考手順は付録 C）。
- ・ **顧客環境との対応（確認済み事項）**：
 - 前段=IIS、SP=Shibboleth SP（IIS ネイティブモジュール）、PLM=IIS 上のアプリ（顧客と一致）。
 - IdP=Entra ID（+独自認証）。識別子は **emailAddress 形式**（例 01PLM01@plm-lab.local）。
 - PLM は将来、メール形式の識別子を受け取り @ の前を切り出して従来の識別番号として扱う想定（**PLM アプリ側の責務**であり、本書の SP/IdP 構築範囲外）。

⚠ 重要な前提：本環境は Windows 仮想マシン 1 台にすべてが同居します。仮想マシンを削除すると IdP/LDAP/SP すべてが失われます。評価版の期限は削除・再インストールではなく slmgr /rearm で延長してください（付録 A）。作業成果の保全・再構築検証の考え方は付録 B を参照。

2. 全体アーキテクチャ





SAML Web SSO の流れ（未認証時）：

1. ブラウザが IIS (SP・443) の保護対象 (PLM 相当) にアクセスする。
2. SP が「未認証」と判断し、ブラウザを IdP (8443) へリダイレクトする。
3. ブラウザが IdP (Tomcat・8443) のログイン画面にアクセスする。
4. IdP が ApacheDS (LDAP・389) に問い合わせてユーザーを認証する (サーバ間通信)。
5. IdP が SAML アサーションを、ブラウザ経由で SP の ACS へ POST する。
6. SP がアサーションを検証してセッションを確立し、REMOTE_USER を確定する。
7. IIS が wwwroot 配下の PLM 相当アプリを実行して応答する。

WSL 版との違い：IdP 側が Windows ネイティブのため、**IdP 前段の Apache HTTPD は不要** (Tomcat が HTTPS 8443 を直接公開)。WSL2 の mirrored ネットワーク・localhost 転送・ポート衝突といった問題は発生しない。IIS (SP) と Tomcat (IdP) は同一 Windows 上に同居するが、**互いに直接は通信せず**、ブラウザのリダイレクトと事前交換したメタデータで連携する。LDAP を参照するのは IdP のみ。

3. パラメーター一覧（確定・調整用）

本書全体で参照する値。**着手前に内容を確認し**、変更する場合はここを起点に各フェーズへ反映してください。

項目	既定値	備考
内部ドメイン	plm-lab.local	検証用の架空ドメイン
IdP ホスト名	idp.plm-lab.local	Tomcat (IdP) 側
SP ホスト名	sp.plm-lab.local	IIS (SP) 側
Java	OpenJDK 17 (Windows 版)	Shibboleth IdP 5 の要件。Temurin 等
Servlet コンテナ	Apache Tomcat 10.1.x (Windows サービス)	IdP 5 は Tomcat 10.1 必須 (9 系不可)
IdP	Shibboleth IdP 5.x	Java 17 / Jakarta 名前空間
SP	Shibboleth SP 3.x (IIS ネイティブモジュール)	IIS 用

項目	既定値	備考
	ジュール)	
LDAP	ApacheDS (Java 製・Windows サービス)	Apache Directory Studio で投入。inetOrgPerson
IdP entityID	https://idp.plm-lab.local/idp/shibboleth	フェーズ 5 で確定
SP entityID	https://sp.plm-lab.local/shibboleth	フェーズ 8 で確定
LDAP ベース DN	dc=plm-lab,dc=local	ApacheDS (フェーズ 3)
LDAP 検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	IdP の LDAP 検索用 (読取専用)
テストユーザー	uid=01PLM01 / uid=01PLM02	ou=people 配下。識別番号 (社内テスト環境の形式に準拠)
テストユーザーの mail	01PLM01@plm-lab.local / 01PLM02@plm-lab.local	emailAddress 形式 NameID の元 (顧客 Entra の形式に準拠)
NameID 形式	emailAddress	顧客 (Entra ID) の形式に合わせる (WSL 版の unspecified から変更)
REMOTE_USER	優先リスト (例 azure_id … gid_id に倣う)	メール形式 NameID を載せる。学習では 1~2 個で可
IdP ホーム	C:\opt\shibboleth-idp	Shibboleth IdP 5 (Windows・フェーズ 5)
Tomcat ホーム	C:\opt\tomcat (例)	Windows サービスとして稼働 (フェーズ 4)
SP インストール先	C:\opt\shibboleth-sp	既定。C:\inetpub 配下は不可 (鍵・設定の漏洩リスク)
PLM アプリ配置	C:\inetpub\wwwroot 配下	検証では REMOTE_USER 表示のテストページ
SP ポート (HTTPS)	443	IIS (SP)
IdP ポート (HTTPS)	8443	Tomcat (IdP) 直公開。同一 Windows 上での 443 衝突回避
内部 CA ルート名	PLM-Lab Root CA	フェーズ 2 で作成。idp/sp のサーバ証明書を発行
証明書作成ツール	openssl for Windows (Git for Windows 同梱等)	WSL 版の openssl 手順を流用
ゲスト VM メモリ	8 GB (目安)	フルスタック同居のため
ゲスト VM vCPU	4	目安
タイムゾーン	Asia/Tokyo	ゲスト Windows

ホスト名は当面 hosts ファイルで解決します（DNS サーバは立てません）。SP・IdP とともにゲスト Windows 上のため、127.0.0.1 で解決させます（フェーズ 2）。

2 種類の証明書（再掲・重要）：(a) TLS/HTTPS サーバ証明書（ブラウザに信頼される・SAN 一致必須。内部 CA で idp/sp を発行）と、(b) SAML 署名・暗号化証明書（メタデータ交換で信頼・自己署名可・CA 信頼もホスト名一致も不要。IdP はインストール時に自動生成、SP は keygen で生成）。準備が要るのは (a)。詳細はフェーズ 2 で扱う。

3.1 アカウント・パスワード一覧（わかりやすさ優先）

本検証環境では**わかりやすさを優先**し、既定値や「Joe アカウント（uid とパスワードが同じ）」を採用する。**いずれも検証専用であり、実際の環境構築時にはセキュリティを考慮して強固な値に変更すること。**OS ユーザーとしての専用アカウント（WSL 版の tomcat 相当）は Windows では作らず、各サービスは Windows のサービスアカウント（Local System 等）で動作する。構築作業はローカル管理者（本書では Administrator。読み替え可）で、昇格した PowerShell／コマンドプロンプトで行う。

#	アカウント／鍵	値（検証用）	使用箇所	フェーズ
1	ApacheDS 管理者	uid=admin,ou=system / secret	ApacheDS 管理（既定値のまま）	3
2	テストユーザー 1	uid=01PLM01 / パスワード 01PLM01	ログイン検証（Joe アカウント）	3
3	テストユーザー 2	uid=01PLM02 / パスワード 01PLM02	2 人目・再現性検証（Joe アカウント）	3
4	IdP 検索バインド	uid=idp-reader / パスワード idp-reader	IdP の LDAP 検索（読取専用）	3・5
5	IdP キーストア	changeit	IdP install.bat 対話	5
6	IdP Sealer	changeit	IdP install.bat 対話	5
7	TLS 証明書 PFX（idp/sp）	changeit	idp.pfx / sp.pfx の作成・取込（Tomcat・IIS）	2・6・7

実機で使ったインストーラ／パッケージ（検証時点の版。最新版があれば読み替え可）：OpenJDK17U-jdk_x64_windows_hotspot_17.0.19_10.zip（Temurin 17）、apacheds-2.0.0.AM27.exe（ApacheDS）、ApacheDirectoryStudio-2.0.0.v20210717-M17-win32.win32.x86_64.zip（Directory Studio）、apache-tomcat-10.1.57-windows-x64.zip（Tomcat）、shibboleth-identity-provider-5.2.3.zip（IdP）、shibboleth-sp-3.5.2.3-win64.msi（SP）。

OS ユーザーの扱い：構築は Administrator（ローカル管理者）。Tomcat/IdP・ApacheDS・shibd（Shibboleth Daemon）はいずれも Windows サービスとして Local System 等のサービスアカウントで動作するため、Linux の tomcat ユーザーのような専用 OS ユーザーの作成・所有権付与は不要。

4. 構築フェーズ全体像（ロードマップ）

フェーズ	内容	WSL 版との関係	本書での状態
1	検証環境の 初期設定 （ゲスト W indows V M・時刻同 期・前提）	新規（WSL 導入を廃し簡素化）	✓ 本版で記載
2	ネットワー ク土台（ho sts・内部 C A・idp/sp 証明書 (a)・ポー ト設計）	変更（openssl for Windows で流 用）	✓ 本版で記載
3	ApacheDS （LDAP） 導入・ディ レクトリ設 計・テスト ユーザー投 入	新規（OpenLDAP から置換）	✓ 本版で記載
4	OpenJDK + Tomcat（W indows サー ビス）	変更（Windows 版・service.bat）	✓ 本版で記載
5	Shibboleth I dP 5（Wind ows・LDAP 連携・emai lAddress Na meID 準 備）	変更（install.bat・Windows パス）	✓ 本版で記載
6	Tomcat 直 H TTPS（844 3）公開	置換（Apache 前段を廃止）	✓ 本版で記載
7	IIS（SP の 保護対象サ イト・443/ TLS）	流用（WSL 版とほぼ同一）	✓ 本版で記載
8	Shibboleth S P（IIS ネイ ティブモ ジュール・ サイト全体 保護）	流用（WSL 版とほぼ同一）	✓ 本版で記載

フェーズ	内容	WSL 版との関係	本書での状態
9	メタデータ 交換 (IdP → SP の相 互信頼・初 回 SSO)	変更 (:8443 補正が不要に)	✅ 本版で記載
10	属性連携 (emailAdd ress 形式 N ameID を R EMOTE_USE R に載せ る)	変更 (unspecified→emailAddress)	✅ 本版で記載
11	結合テスト (ログイ ン・再現 性・再起動 堅牢性・ロ グ)	流用／変更	❌ 未

各フェーズは「目的 → 前提 → 手順 → 動作確認」の順で記載します。付録：A（評価版 rearm）／B（バックアップ・再現性検証のチェックポイント運用）／C（スリープ環境向け時刻再同期）／D（トラブルシュート・Windows 固有）／E（オフライン導入）。

5. フェーズ 1：検証環境の初期設定

目的：Hyper-V ゲストの Windows 11 を、以降の全スタック（ApacheDS／Tomcat+IdP／IIS+SP）を同居させる土台として整える。あわせて、SAML でつまずきやすい**時刻同期**を最初に固め、やり直しに備えた**着手前チェックポイント**を取得する。

WSL 版との違い：本構成は WSL2 を使わないため、**入れ子（ネステッド）仮想化の有効化は不要**。ゲスト Windows 上で Java/Tomcat/IIS が動くだけなので、通常の Hyper-V ゲストとして扱える（WSL 版 \$5.2 の ExposeVirtualizationExtensions や MAC スプーフィングは本版では不要）。

5.1 事前確認

- ・ 検証用 PC で Hyper-V が有効で、ゲストの Windows 11 評価版が作成済みであること。
- ・ ゲストに割り当てるメモリ（8GB 目安）・vCPU（4 目安）の余裕が検証用 PC にあること。
- ・ ゲスト Windows に**ローカル管理者（Administrator。自宅では読み替え）**でサインインできること。
- ・ 物理ホスト・ゲストとも **Windows のスリープ／休止が無効**であること（\$1 の前提）。

5.2 ゲスト VM の電源オプションと着手前チェックポイント

やり直しに備え、**素の状態のチェックポイント**を取得します。まず、電源断時に保存状態へ入らずシャットダウンさせる設定にしてから、VM を停止した状態でチェックポイントを取得します。検証用 PC（物理ホスト）の**管理者権限 PowerShell**で実行します。

1) 対象 VM 名を確認

Get-VM

2) 電源断時に保存状態にせずシャットダウンさせる（保存状態からの復帰による時刻ずれ回避）

Set-VM -Name "<VM名>" -AutomaticStopAction ShutDown

3) 種類を標準（Standard）チェックポイントに（本構成は入れ子仮想化を使わないため Standard で可）

Set-VM -Name "<VM名>" -CheckpointType Standard

4) ゲストをシャットダウン（停止状態で取得するのが最も安全）

Stop-VM -Name "<VM名>"

5) 着手前の素の状態を取得

Checkpoint-VM -Name "<VM名>" -SnapshotName "Phase1 前_素の Windows11"

⚠ パラメータ名の注意（-Name と -VMName）：VM 自体を操作する系（Set-VM/Get-VM/Start-VM/Stop-VM/Checkpoint-VM）は VM 名を -Name で指定する。VM の構成要素を操作する系（Set-VMProcessor/Set-VMMemory 等）は -VMName だが、本フェーズでは使用しない。取り違えを避けたい場合は Get-VM "<VM名>" | Set-VM -CheckpointType Standard のようにパイプで渡す。

チェックポイントは短期のやり直し用でありバックアップではありません（付録 B）。本構成は入れ子仮想化を使わないため、標準（Standard）チェックポイントが利用でき、稼働中の取得も比較的安全ですが、着手前の素の状態は**停止して**取得するのが確実です。

5.3 時刻同期の確認

SAML は IdP と SP の時刻の一致に敏感で、ズレると「clock skew」エラーの原因になります。本構成では IdP（Tomcat）も SP（IIS）も**同一のゲスト Windows 上**にあるため、両者の時刻は常に一致し、WSL 版のような OS 間の相対ずれは原理的に発生しません。したがって、ゲスト Windows の時計が正しく保たれていれば十分です。

- ・ **Hyper-V 統合サービス「時刻同期」が有効**であることを確認（既定で有効）。これによりゲスト Windows は検証用 PC（ホスト）の時計に追従します。ゲストの管理者 PowerShell で確認：

タイムゾーンが東京か

Get-TimeZone

東京でなければ設定

Set-TimeZone -Name "Tokyo Standard Time"

Windows Time サービスが稼働しているか（オンライン時は NTP 同期も有効）

w32tm /query /status

オンライン環境なら Windows Time サービス（w32tm）が NTP に同期します。オフライン環境でも、IdP と SP が同一 OS 上のため相対ずれは発生せず、SAML の clock skew は問題になりません。スリープを使う環境で運用する場合の復帰時再同期は付録 C を参照。

5.4 導入方針と作業用フォルダの準備

導入方針（本書全体）：zip パッケージが選べるものは zip を展開して導入し、環境変数など必要な設定は手作業で行う（どこに何が入ったかが明確になり、再現・撤去・切り分けが容易）。導入物は原則 C:\opt 配下・空白を含まないパスに統一する（Java アプリでの空白起因トラブルを避ける）。作業は昇格した PowerShell/コマンドプロンプトで行う。

例外：ApacheDS 本体は zip 提供が無いためインストーラ（.exe）で導入する（\$7.2）。

作業用フォルダを用意します。

`New-Item -ItemType Directory -Force -Path C:\opt, C:\lab, C:\lab\ca, C:\lab\installers | Out-Null`

- C:\opt：jdk-17／directory-studio／tomcat／shibboleth-idp／shibboleth-sp の導入先。
- C:\lab\ca：内部 CA・証明書の作成場所（フェーズ 2）。
- C:\lab\installers：入手した各インストーラ／zip の保管。

5.5 動作確認チェックリスト（フェーズ 1）

#	確認内容	コマンド / 方法	期待結果
1	ゲストにローカル管理者でサインイン	—	Administrator（読み替え可）でサインイン済み
2	タイムゾーン	Get-TimeZone	Tokyo Standard Time
3	時刻サービス	w32tm /query /status	稼働（オンライン時は NTP 同期）
4	着手前チェックポイント	Hyper-V マネージャー	Phase1 前_素の Windows11 が存在
5	昇格実行の確認	管理者 PowerShell が使える	昇格プロセスでコマンド実行可

5.6 OpenJDK（Temurin 17）の導入（ApacheDS・Tomcat の前提）

ApacheDS（フェーズ 3）と Tomcat（フェーズ 4）はいずれも Java で動作する。**Java は両者の共通基盤のため、ここ（フェーズ 3 の前）で先に導入しておく**（ApacheDS インストーラが JAVA_HOME を求めるため、順序として Java が先）。

1. **入手**：Adoptium (<https://adoptium.net/>) から **Temurin 17（LTS）・Windows x64・JDK・zip** を入手し C:\lab\installers へ。
2. **展開&リネーム**（管理者 PowerShell）：

```
Expand-Archive -Path "C:\lab\installers\OpenJDK17U-jdk_x64_windows_hotspot_*.zip" -DestinationPath "C:\opt" -Force
Get-ChildItem C:\opt | Where-Object Name -like "jdk-17*"           # 展開フォルダ名を確認
Rename-Item "C:\opt\jdk-17.0.xx+xx" "C:\opt\jdk-17"             # 分かりやすく（任意）
```

3. **環境変数（システム）を手動設定**：

```
[Environment]::SetEnvironmentVariable("JAVA_HOME", "C:\opt\jdk-17", "Machine")
$P = [Environment]::GetEnvironmentVariable("Path", "Machine")
[Environment]::SetEnvironmentVariable("Path", "$P;C:\opt\jdk-17\bin", "Machine")
```

設定後は**新しい PowerShell を開き直す**（既存セッションには反映されない）。

4. **確認**：java -version（openjdk version “17...”）、echo \$env:JAVA_HOME。

この後にフェーズ 2（証明書）・フェーズ 3（ApacheDS）へ進む。ApacheDS インストーラが JAVA_HOME を尋ねたら C:\opt\jdk-17 を指定する。

すべて確認できれば、フェーズ1は完了です。次はフェーズ2（ネットワーク土台・内部CAとidp/sp証明書の発行）です。

6. フェーズ2：ネットワーク土台（hosts・内部CA・idp/spサーバ証明書）

目的：コンポーネント導入の前に、全体が依存する「名前解決（hosts）」と「TLS証明書」を先に確定する。純 Windows 構成では SP（IIS）も IdP（Tomcat）も**同一のゲスト Windows 上**にあり、いずれも 127.0.0.1 で解決するため、WSL 版のようなネットワーク方式の選定（NAT／mirrored の段階移行）や、localhost 転送・ポート転送に関する検討は**不要**になる。

WSL 版との違い：WSL 版では「ブラウザ→WSL2 の到達性」を確保するため mirrored モードや (A)→(B) 段階移行を検討したが、本構成では IdP が Windows ネイティブのため、その検討は丸ごと不要。ポート設計（SP=443／IdP=8443）だけは同じ考え方で踏襲する（同一ホスト上で 443 を取り合わないため）。

6.1 ポート設計

同一のゲスト Windows 上で SP（IIS）と IdP（Tomcat）が 443 を取り合わないよう、IdP を別ポート（8443）にします。

役割	ホスト名	URL（ベース）	ポート	バインド先
SP	sp.plm-lab.local	https://sp.plm-lab.local/	443	IIS
IdP	idp.plm-lab.local	https://idp.plm-lab.local:8443/idp/	8443	Tomcat（直 HTTPS）

SAML のエンドポイントはすべて URL なので、ポートが異なっても問題ありません。顧客の本番（IdP=Entra ID）では IdP は Entra 側の URL になるため、この 8443 は「学習用テスト IdP を同一ホストに同居させるための便宜」です。

6.2 名前解決（hosts）

DNS サーバは立てず、hosts で解決します。SP・IdP とも同一のゲスト Windows 上のため、両方を 127.0.0.1 に向けます。

ゲスト Windows：C:\Windows\System32\drivers\etc\hosts（**管理者権限で編集**。昇格したメモ帳等）

```
127.0.0.1 sp.plm-lab.local
127.0.0.1 idp.plm-lab.local
```

管理者 PowerShell で追記する場合：

```
Add-Content -Path "$env:SystemRoot\System32\drivers\etc\hosts" -Value "127.0.0.1`tsplm-lab.local`r`n127.0.0.1`tidp.plm-lab.local"
```

確認：

```
ping sp.plm-lab.local      # 127.0.0.1 に解決されること
ping idp.plm-lab.local    # 127.0.0.1 に解決されること
```

6.3 【参考】この構成で登場する2種類の証明書

Shibboleth では性格の異なる2種類の証明書が登場し、混同しやすいポイントです。役割を分けて理解してください。

観点	(a) TLS/HTTPS サーバ証明書	(b) SAML 署名・暗号化証明書
目的	ブラウザ↔サーバの HTTPS 通信の暗号化とサーバ真正性	SAML メッセージ（アサーション等）の署名・暗号化
使う場所	IIS (SP, 443) 、Tomcat (IdP, 8443)	IdP・SP の SAML 処理内部
誰が検証するか	ブラウザ	相手方の IdP/SP
信頼の確立方法	CA を信頼ストアに登録 (PKI)	メタデータ交換で公開鍵を相互登録
ホスト名 (SAN) 一致	必須	不要
CA 署名	必要（本書では内部 CA で発行）	不要（自己署名が通常）
準備するタイミング	フェーズ 2（本節）	各コンポーネント導入時に生成（IdP=フェーズ 5、SP=フェーズ 8）

本節（6.4）で用意するのは (a) のみ。(b) は後続フェーズでコンポーネントが生成します。両者は独立で、(b) に CA 信頼やホスト名一致は不要です。

6.4 内部 CA の作成と idp/sp サーバ証明書の発行（(a) の準備）

Git for Windows 同梱の openssl を使います。**Git Bash** を起動して以下を実行します（WSL 版と同じ openssl 手順をそのまま流用）。作業場所は \$5.4 で作成した C:\lab\ca（Git Bash では /c/lab/ca）。

```
cd /c/lab/ca
```

1) 内部 CA ルート（秘密鍵と自己署名証明書、10 年）

```
openssl genrsa -out rootCA.key 4096
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 \
  -subj "//C=JP\O=PLM-Lab\CN=PLM-Lab Root CA" -out rootCA.crt
```

2) idp のサーバ証明書

```
cat > idp.ext <<'EOF'
subjectAltName = DNS:idp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
openssl genrsa -out idp.key 2048
openssl req -new -key idp.key -subj "//C=JP\O=PLM-Lab\CN=idp.plm-lab.local" -out idp.csr
openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
  -out idp.crt -days 825 -sha256 -extfile idp.ext
```

3) sp のサーバ証明書

```
cat > sp.ext <<'EOF'
subjectAltName = DNS:sp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
```

```
openssl genrsa -out sp.key 2048
openssl req -new -key sp.key -subj "//C=JP\O=PLM-Lab\CN=sp.plm-lab.local" -out sp.csr
openssl x509 -req -in sp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
-out sp.crt -days 825 -sha256 -extfile sp.ext
```

⚠ **Git Bash 特有の注意（MSYS のパス変換）**：Git Bash では -subj "/C=JP/..." の先頭の / が Windows パスに誤変換されることがあります。上記のように **先頭を //**にし、**区切りを ** ("//C=JP\O=PLM-Lab\CN=...") とすると回避できます。うまくいかない場合は、環境変数 MSYS_NO_PATHCONV=1 を付けて実行（例：MSYS_NO_PATHCONV=1 openssl req ...）してもよいです。

PFX (PKCS12) 化：IIS (sp) と Tomcat (idp) は、HTTPS バインドに証明書＋秘密鍵を **PFX 形式** で取り込みます。両方を作成します（エクスポートパスワードは §3.1 のとおり changeit）。

IIS (フェーズ7) 用

```
openssl pkcs12 -export -out sp.pfx -inkey sp.key -in sp.crt -certfile rootCA.crt -passout pass:changeit
```

Tomcat/IdP (フェーズ6) 用

```
openssl pkcs12 -export -out idp.pfx -inkey idp.key -in idp.crt -certfile rootCA.crt -passout pass:changeit
```

6.4.1 【解説】証明書コマンドの意味

§6.4 の各コマンドが「何をしているか」を、後から理解・説明できるよう解説します。全体像は「**自前の認証局 (CA) を1つ作り、その CA で idp と sp のサーバ証明書に署名する**」という PKI の縮小版です。

3 ステップの関係

- [1] 内部 CA ルートを作る (rootCA.key / rootCA.crt) … 署名する側 (親)
 - └─ [2] idp のサーバ証明書を CA に署名してもらう (idp.key / idp.crt)
 - └─ [3] sp のサーバ証明書を CA に署名してもらう (sp.key / sp.crt)

サーバ証明書 (idp.crt/sp.crt) は単体では信頼されず、**信頼された CA (rootCA) の署名が付いて初めてブラウザに信頼される**。だから先に CA を作り、その CA で各サーバ証明書に署名する。

[1] 内部 CA ルート

- openssl genrsa -out rootCA.key 4096：CA の**秘密鍵**を生成。4096 は鍵長（CA は大元なのでサーバ証明書の 2048 より長く強度を確保）。生成物 rootCA.key は**最重要機密**（これで任意の証明書に署名できる）。
- openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -subj "...CN=PLM-Lab Root CA" -out rootCA.crt：秘密鍵を使って CA 自身の**ルート証明書**を作る（**自己署名**）。
 - -x509：CSR でなく証明書そのものを出力（自己署名になる）。-new：新規。-nodes：秘密鍵をパスフレーズで暗号化しない（起動時のパスワード入力を不要にする）。-key：署名に使う秘密鍵。-sha256：署名ハッシュ。-days 3650：有効期間約 10 年（CA は長寿命）。-subj：サブジェクト（C 国／O 組織／CN＝この CA の名前。Windows の信頼ストアに表示される名前になる）。先頭 // と区切り \ は Git Bash のパス誤変換回避。
 - 生成物 rootCA.crt：CA の公開ルート証明書（**公開可**）。各マシンの信頼ストアに登録して「この CA が署名した証明書を信頼する」状態を作る。

[2] idp のサーバ証明書

- `cat > idp.ext …`：証明書に付ける**拡張**を書いたファイル。subjectAltName = DNS:idp.plm-lab.local (**SAN**。現代のブラウザはCNでなくSANでホスト名一致を判定するため必須)、extendedKeyUsage = serverAuth (**サーバ認証用途**)。
- `openssl genrsa -out idp.key 2048`：idp サーバの**秘密鍵**（サーバ用は2048）。
- `openssl req -new -key idp.key -subj "...CN=idp.plm-lab.local" -out idp.csr`：**CSR（証明書署名要求＝申請書）**を作成。CSRにはidpの**公開鍵**とサブジェクトが入る（この時点ではCA署名なし）。
- `openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out idp.crt -days 825 -sha256 -extfile idp.ext`：**CAが申請書に署名して正式なサーバ証明書を発行**（中核）。
 - `x509 -req`：入力がCSRで、署名して証明書を出すモード。-CA/-CAkey：署名に使うCA証明書とCA秘密鍵。-CAcreateserial：シリアル番号管理ファイル（rootCA.srl）を作成/更新。-days 825：有効期間（ブラウザ制限に合わせ短め）。-extfile idp.ext：SAN・用途を証明書に付与（**忘れるとホスト名不一致警告**）。
 - 生成物 idp.crt：idpの公開鍵＋サブジェクト＋SAN＋**CAの署名**が入った正式なサーバ証明書。

[3] sp のサーバ証明書

[2] の idp を sp に置き換えただけで意味は同じ（SAN=DNS:sp.plm-lab.local、CN=sp.plm-lab.local）。**同じCA（rootCA）で署名**するため、rootCAを1つ信頼登録すればidp・sp両方が信頼される。

生成ファイルまとめ

ファイル	種類	機密性	用途
rootCA.key	CA 秘密鍵	最重要機密	各サーバ証明書への署名
rootCA.crt	CA ルート証明書	公開可	信頼の起点。各マシンの信頼ストアに登録
rootCA.srl	シリアル管理	—	証明書ごとの通し番号（自動）
idp.key / sp.key	サーバ秘密鍵	機密	Tomcat(IdP)・IIS(SP)のTLS復号
idp.csr / sp.csr	署名要求	—	発行時の中間ファイル
idp.ext / sp.ext	拡張設定	—	SAN・用途を付与する中間ファイル
idp.crt / sp.crt	サーバ証明書	公開可	Tomcat(IdP,8443)・IIS(SP,443)のHTTPS
idp.pfx / sp.pfx	証明書＋秘密鍵(PKCS12)	機密（鍵を含む）	Tomcat・IISへのインポート用

要点：CAを自前で1つ作りrootCAを各マシンに登録すれば、内部のサーバ証明書がすべて信頼される。手順は「秘密鍵→CSR－CA署名→証明書」で、公的証明書取得の縮小版。

SAN が無いと鍵マークにならないため subjectAltName が肝。鍵長は CA=4096 / サーバ=2048 と使い分ける。ここで作るのは (a) TLS/HTTPS サーバ証明書のみで、(b) SAML 署名・暗号化証明書はフェーズ 5・8 でコンポーネントが生成する (別物)。

6.5 rootCA をゲスト Windows の信頼ルートに登録

ブラウザ (Edge/Chrome は Windows 証明書ストアを使用) で鍵マークにするため、内部 CA ルートを「信頼されたルート証明機関」に登録します。管理者 PowerShell で：

```
Import-Certificate -FilePath "C:\lab\ca\rootCA.crt" -CertStoreLocation Cert:\LocalMachine\Root
```

別マシン (検証用 PC=ホストや LAN 上の PC) のブラウザからアクセスする「発展編」を行う場合は、そのマシンの証明書ストアにも同じ rootCA.crt を登録すれば、ホスト名にひも付く証明書はそのまま使えます (作り直し不要)。ただし本書の基本構成はゲスト Windows 上のブラウザで検証します。

6.6 動作確認チェックリスト (フェーズ 2)

#	確認内容	方法	期待結果
1	hosts 解決	ping idp.plm-lab.local / ping sp.plm-lab.local	いずれも 127.0.0.1
2	CA・証明書生成	ls /c/lab/ca (Git Bash)	rootCA.crt / idp.crt / sp.crt / idp.pfx / sp.pfx 等が存在
3	SAN の確認	openssl x509 -in idp.crt -noout -text \ grep -A1 "Subject Alternative"	DNS:idp.plm-lab.local
4	証明書の検証	openssl verify -CAfile rootCA.crt idp.crt / sp.crt	OK
5	Windows の CA 信頼	certlm.msc - 「信頼されたルート証明機関」	PLM-Lab Root CA が存在

この時点では証明書を使う Web サーバ (Tomcat/IIS) がまだ無いため、HTTPS 応答の確認は各コンポーネント導入後 (フェーズ 6・7) に行います。

7. フェーズ 3 : ApacheDS (LDAP ユーザーディレクトリ)

目的：IdP が認証・属性取得を行うユーザーの元データを、**ApacheDS** (Java 製 LDAP サーバ・Windows で動作) に用意する。ディレクトリ設計 (baseDN・ou=people・テストユーザー・検索バインド) は WSL 版の OpenLDAP と同じ考え方を踏襲し、製品を ApacheDS に置き換える。ユーザー投入は **Apache Directory Studio** (GUI) または **LDIF インポート**で行う。

WSL 版との違い：OpenLDAP (apt・slapd・dpkg-reconfigure・LDIF+ldapadd) から、ApacheDS (Windows インストーラ・GUI ツール) に置き換わる。設計値 (dc=plm-lab,dc=local/ou=people/inetOrgPerson/uid・mail・userPassword) は同じ。メール形式の識別子に対応するため、各ユーザーに **mail 属性** (01PLM01@plm-lab.local) を持たせる点が新しい。

7.1 ディレクトリ設計

dc=example,dc=com	← ベース DN (ApacheDS 既定パーティションを流用)
├── ou=people	← ユーザー・検索アカウント
├── uid=01PLM01 (inetOrgPerson)	← テストユーザー 1 (mail: 01PLM01@plm-lab.local)
├── uid=01PLM02 (inetOrgPerson)	← テストユーザー 2 (mail: 01PLM02@plm-lab.local)
└── uid=idp-reader (inetOrgPerson)	← IdP の検索バインド用 (読取)
└── (将来) ou=groups	← ロール等 (本書では未使用)

項目	値
パーティション suffix (baseDN)	dc=example,dc=com (ApacheDS 既定パーティションを流用。7.4 参照)
ユーザー OU	ou=people,dc=example,dc=com
テストユーザー	uid=01PLM01 / uid=01PLM02 (objectClass: inetOrgPerson)。パスワード=uid (J oe)
テストユーザーの mail	01PLM01@plm-lab.local / 01PLM02@plm-lab.local (emailAddress NameID の元)
検索バインド	uid=idp-reader,ou=people,dc=example,dc=com (パスワード idp-reader・読取専用)
ApacheDS 管理者	uid=admin,ou=system (既定パスワード secret)
LDAP ポート	10389 (ApacheDS 既定。IdP 側は ldap://localhost:10389 で参照)

baseDN について (実機の判断) : ApacheDS のインストーラ版 (2.0.0.AM 系) は設定を config.ldif ではなく LDAP 内部 (ou=config) で保持し、dc=plm-lab,dc=local パーティションの手作業追加は難度が高い (7.4 参照)。本書では**既定パーティション dc=example,dc=com をそのまま流用**する (学習・SSO 動作には影響しない)。WSL 版の dc=plm-lab,dc=local からの読み替え表は 7.4 に示す。mail は @plm-lab.local のまま (メールアドレスと baseDN は別物なので揃える必要はない)。

ポートの注意 : ApacheDS の LDAP ポートは実機で **10389** (非特権ポート)。Directory Studio の New Connection では Port 入力欄の既定表示が **389** になることがあるが、実機の待受は 10389 なので **10389 に合わせる** (netstat -ano | findstr 10389 で確認)。WSL 版は 389 だったが、本書では 10389 を用い、フェーズ 5 の IdP 設定 (ldapURL) も ldap://localhost:10389 とする。

7.2 ApacheDS のインストール

前提 (Java を先に導入) : ApacheDS は Java で動作し、インストーラの途中で **JAVA_HOME の入力**を求められる。そのため **OpenJDK (Temurin 17)** を **ApacheDS より前に導入**しておく (\$5.6 参照)。インストーラで JAVA_HOME を聞かれたら、JDK のルート (例 C:\opt\jdk-17。直下に bin\java.exe があるフォルダ。bin は含めない) を指定する。

1. Apache Directory の公式サイト (<https://directory.apache.org/apacheds/download/download-windows.html>) から、**ApacheDS の Windows インストーラ (.exe・最新安定版)** を入手し、C:\lab\installers に保存。※ ApacheDS 本体はインストーラ (.exe) で導入する (zip 提供が無いため。zip 優先方針の例外)。

2. インストーラを**管理者として実行**。JAVA_HOME に上記 JDK を指定し、他は既定のまま進める（実機の導入先は C:\Program Files (x86)\ApacheDS、インスタンスは ...\instances\default）。導入すると **ApacheDS の Windows サービス**（既定インスタンス名 default）が登録される。
3. サービスを起動・確認：

```
Get-Service | Where-Object { $_.DisplayName -like "*ApacheDS*" }  
Start-Service <サービス名> # 停止していれば開始（サービス名は環境で異なる）  
# LDAP 待受ポートの確認（実機は 10389）  
netstat -ano | findstr 10389
```

実機では LDAP は **10389** で待ち受ける（0.0.0.0:10389 LISTENING）。

7.3 Apache Directory Studio の導入と接続

1. 公式サイト（<https://directory.apache.org/studio/download/download-windows.html>）から **Apache Directory Studio (Windows 版・zip)** を入手し、C:\opt\directory-studio に展開する（本書の zip 優先方針）。
2. **起動**：zip 展開ではインストールされず、Windows スタートメニューにも登録されない。エクスプローラーまたはコマンドから C:\opt\directory-studio\ApacheDirectoryStudio.exe を実行して起動する（よく使う場合はショートカットを作成）。
 - Java が見つからず起動しない場合は、C:\opt\directory-studio\ApacheDirectoryStudio.ini に -vm と C:\opt\jdk-17\bin\javaw.exe の 2 行を追記する（通常は不要。JAVA_HOME 設定済みのため）。
3. 起動後、**LDAP 接続を作成**：メニュー **LDAP → New Connection**。
 - Network Parameter：Hostname localhost / Port **10389**（入力欄の既定表示が 389 の場合は 10389 に変更） / Encryption method **No encryption**（同一ホスト通信のため平文で可）
 - Connection name（任意）：管理作業用なので plm-lab-admin など用途が分かる名前を推奨
 - 次画面（Authentication）：Bind DN uid=admin,ou=system / Bind password secret
 - **Check Authentication** を押し、successful を確認してから **Finish**。接続が緑色になれば成功。
4. 左の **LDAP Browser** で、既定パーティション dc=example,dc=com や ou=system・ou=config が見えることを確認。

7.4 パーティション (baseDN) の方針：既定パーティション dc=example,dc=com を流用

当初は WSL 版に合わせ dc=plm-lab,dc=local パーティションを新規作成する想定だったが、**実機のインストーラ版 ApacheDS (2.0.0.AM 系)** では手作業での新規作成は現実的でないことが判明した。理由：

- このバージョンは設定を config.ldif ファイルではなく **LDAP 内部 (ou=config)** として保持する（実際 C:\Program Files (x86)\ApacheDS\instances\default\配下に config.ldif は存在せず、partitions\に example・system の DB がある）。パーティション追加は停止→設定エントリ (ads-partition... 階層) を正確に追記→再起動が必要で、書式ミスで**サービスが起動不能になるリスク**がある。
- Directory Studio に ApacheDS を「サーバ」として登録していない（接続のみ）ため、**LDAP Servers ビューの Open Configuration によるパーティション編集が使えない**。

そこで本書は、**ApacheDS の既定パーティション dc=example,dc=com をそのまま流用**する。パーティション作成は不要で、その配下に ou=people とユーザーを作る。学習・SSO 動作には一切影響しない。

baseDN 読み替え表（当初の plm-lab 版 → 本書の既定流用版）

項目	当初（WSL 版踏襲）	本書（既定流用）
baseDN	dc=plm-lab,dc=local	dc=example,dc=com
ユーザー OU	ou=people,dc=plm-lab,dc=local	ou=people,dc=example,dc=com
テストユーザー	uid=01PLM01,ou=people,dc=plm-lab,dc=local	uid=01PLM01,ou=people,dc=example,dc=com
検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	uid=idp-reader,ou=people,dc=example,dc=com
mail（変更なし）	01PLM01@plm-lab.local	01PLM01@plm-lab.local

どうしても dc=plm-lab,dc=local を作りたい場合は、ApacheDS の設定（ou=config の ou=partitions）にパーティション定義を追加し、コンテキストエントリ（dc=plm-lab,dc=local）を投入する必要がある（バージョン依存・要サービス再起動）。本書では扱わない。

7.5 OU・テストユーザー・検索バインドの投入（LDIF）

Directory Studio で **LDAP メニュー → New LDIF File** を開き、以下を貼り付けて、エディタ右上の緑の ▶（Execute LDIF）で実行します（対象接続は管理者の plm-lab-admin）。dc=example,dc=com 自体は既存のため LDIF に含めず、その配下だけ作ります。パスワードは分かりやすさ優先の平文で記載します（ApacheDS が格納時に自動で SSHA ハッシュ化するため、WSL 版の slapasswd に相当する作業は不要）。

```
# ユーザー OU（既存の dc=example,dc=com 配下に作成）
dn: ou=people,dc=example,dc=com
objectClass: top
objectClass: organizationalUnit
ou: people

# テストユーザー 1（個人番号 01PLM01・Joe アカウント）
dn: uid=01PLM01,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: 01PLM01
cn: Test User 01PLM01
sn: 01PLM01
mail: 01PLM01@plm-lab.local
userPassword: 01PLM01

# テストユーザー 2（個人番号 01PLM02・Joe アカウント）
dn: uid=01PLM02,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: 01PLM02
cn: Test User 01PLM02
sn: 01PLM02
mail: 01PLM02@plm-lab.local
```

```
userPassword: 01PLM02
```

```
# IdP 検索バインド用（読取専用アカウント）
dn: uid=idp-reader,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: idp-reader
cn: IdP Reader
sn: Reader
userPassword: idp-reader
```

実行前は未投入：LDIF タブ名が *LDIF ...（先頭に*）の間は編集集中で、サーバへは未送信。
右上の緑の ►（Execute LDIF） を押して初めて反映される。成功すると下部 **Modification Logs** に #!RESULT OK と各 add が記録され、左ツリーの dc=example,dc=com を Reload すると ou=people 配下が現れる。userPassword は Entry editor で「SSHA hashed password」と表示され、平文が自動ハッシュ化されたことを確認できる。

objectClass の階層：inetOrgPerson は organizationalPerson → person → top を継承するため、上記のように4つを併記します。person の必須属性 cn・sn を必ず与えます（Directory Studio の GUI 追加でも同様に求められます）。

GUI で投入する場合：LDAP Browser で dc=example,dc=com を右クリック → **New Entry - Create entry from scratch** で ou=people（organizationalUnit）を作成 → 続けて ou=people の下に objectClass inetOrgPerson のエントリを作成、RDN を uid=01PLM01、cn・sn・mail を入力し、最後に **New Attribute** で userPassword を追加。

7.6 動作確認（検索バインド）

投入後、**検索バインド（idp-reader）でテストユーザーを引けるか**を確認します。これはフェーズ5で IdP が行う動作の先取り確認です。

方法1：Directory Studio で確認 - 新しい接続（例：plm-lab-idp-reader）を Port 10389 / Encryption No encryption / Bind DN uid=idp-reader,ou=people,dc=example,dc=com / パスワード idp-reader で作成し、**Check Authentication** が successful になること。 - LDAP Browser で ou=people を展開し、uid=01PLM01・01PLM02 が見えること（mail 属性に 01PLM01@plm-lab.local 等が入っていること）。

方法2：検索で確認（推奨） Directory Studio の検索機能で、検索ベース ou=people,dc=example,dc=com、フィルタ (uid=01PLM01) を実行し、1件返ることを確認。

#	確認内容	期待結果
1	ApacheDS サービス稼働	ApacheDS サービスが実行中、LDAP ポート 10389 で待受
2	管理者接続	uid=admin,ou=system / secret で接続可
3	ベース DN	既定パーティション dc=example,dc=com を使用
4	エントリ投入	ou=people,dc=example,dc=com 配下に 01PLM01 / 01PLM02 / idp-reader
5	検索バインド	uid=idp-reader,ou=people,dc=example,dc=com で bind（Check Authentication successful）し、(uid=01PLM01) が1件返る

#	確認内容	期待結果
6	mail 属性	01PLM01 の mail が 01PLM01@plm-lab.local
7	パスワード格納	userPassword が「SSHA hashed password」 (平文が自動ハッシュ化)

フェーズ5のIdPは、このidp-readerでLDAPに検索バインドし、ログインユーザーのuidで認証、mailをNameID(emailAddress形式)の元として取得します。フェーズ5のIdP設定でそのまま使う値：**LDAP URL** ldap://localhost:10389/**検索ベース** ou=people,dc=example,dc=com/**検索バインド** uid=idp-reader,ou=people,dc=example,dc=com/**パスワード** idp-reader/**フィルタ** (uid={user})/**NameIDの元** mail。

8. フェーズ4：OpenJDK + Tomcat（Windows サービス）

目的：Shibboleth IdP（フェーズ5）を載せる土台として、Tomcat 10.1を Windows サービスとして稼働させる。OpenJDK（Temurin 17）は \$5.6 で導入済みのため、本フェーズは Tomcat に集中する。まず HTTP 8080 で起動確認し、HTTPS 8443 化はフェーズ6で行う。

前提：java -version が 17 を返し、JAVA_HOME=C:\opt\jdk-17 が設定済みであること（\$5.6）。
IdP 5 は **Tomcat 10.1 系が必須**（9 系や 11 は不可。Jakarta 名前空間）。

8.1 Tomcat 10.1 の入手と展開（zip）

1. Apache Tomcat 公式（<https://tomcat.apache.org/download-10.cgi>）から、**Tomcat 10.1.x の「Core」zip**（例 apache-tomcat-10.1.xx-windows-x64.zip または apache-tomcat-10.1.xx.zip）を入手し C:\lab\installers へ。
2. C:\opt\tomcat に展開（zip 内のトップフォルダを C:\opt\tomcat に合わせる）。管理者 PowerShell：

```
Expand-Archive -Path "C:\lab\installers\apache-tomcat-10.1.*.zip" -DestinationPath "C:\opt" -Force
# 展開されたフォルダ名を確認（例：apache-tomcat-10.1.56）
Get-ChildItem C:\opt | Where-Object Name -like "apache-tomcat-*"
# C:\opt\tomcat にリネーム（bin/conf/lib などが直下に来るように）
Rename-Item "C:\opt\apache-tomcat-10.1.xx" "C:\opt\tomcat"
# 確認：bin\startup.bat 等が見えること
Get-ChildItem C:\opt\tomcat\bin\startup.bat, C:\opt\tomcat\conf\server.xml
```

8.2 環境変数（CATALINA_HOME）の設定

Tomcat の場所を示す CATALINA_HOME をシステム環境変数に設定します（サービス化・起動スクリプトが参照）。管理者 PowerShell：

```
[Environment]::SetEnvironmentVariable("CATALINA_HOME", "C:\opt\tomcat", "Machine")
```

設定後は**新しい PowerShell／コマンドプロンプトを開き直す**（既存セッションには反映されない）。JAVA_HOME は \$5.6 で設定済みのため、Tomcat のサービスも JVM を見つけられる。

8.3 Windows サービスとして登録（service.bat）

zip 版には bin\service.bat（サービス登録スクリプト）が含まれます。**管理者コマンドプロンプト（PowerShell ではなく cmd を推奨。service.bat はバッチのため）**で実行します。

```
cd /d C:\opt\tomcat\bin
service.bat install
```

- 成功すると、サービス名 **Tomcat10**（既定。サービスの表示名は実機で「**Apache Tomcat 10.1 Tomcat10**」となる）が登録されます（`service.bat install <名前>` で任意名も可）。
- UAC が有効な場合、Tomcat10.exe 起動時に追加の権限を求められることがあります（管理者で実行していれば通過）。

JVM (Java) をサービスに確実に認識させる：サービスが JVM を見つけられないと起動に失敗します。GUI 設定ツール `tomcat10w` で確認・調整できます。

```
C:\opt\tomcat\bin\tomcat10w.exe //ES//Tomcat10
```

- 開いたダイアログの **Java** タブで、**Java Virtual Machine** が `C:\opt\jdk-17\bin\server\jvm.dll`（または `...\bin\jvm.dll`）を指しているか確認。空欄や誤りがあれば設定。
- **Startup type** を **Automatic**（自動起動）にしておく、Windows 起動時に Tomcat も上がる（再起動堅牢性）。

`service.bat install` 実行時に `JAVA_HOME` が正しく通っていれば、JVM は自動設定されることが多いです。起動に失敗する場合はまずこの `tomcat10w` の Java タブを確認します。

8.4 起動と動作確認 (8080)

```
Start-Service Tomcat10
```

```
Get-Service Tomcat10 # Status が Running
```

```
# HTTP 応答 (既定コネクタは 8080)
```

```
Invoke-WebRequest http://localhost:8080/ -UseBasicParsing | Select-Object StatusCode
```

```
# 待受ポート確認
```

```
netstat -ano | findstr 8080
```

- ブラウザで `http://localhost:8080/` を開き、Tomcat の初期ページが表示されれば成功。
- ログは `C:\opt\tomcat\logs\` (`catalina.*.log`) で確認できます。

補足 (8080 の公開範囲)：本構成では IdP へのブラウザアクセスはフェーズ 6 で HTTPS 8443 経由にするため、8080 は最終的に localhost 限定でも構いません (`server.xml` の HTTP コネクタに `address="127.0.0.1"` を付けて絞れる)。フェーズ 4 の時点では既定（全インターフェース）で確認して問題ありません。

8.5 動作確認チェックリスト (フェーズ 4)

#	確認内容	コマンド / 方法	期待結果
1	Java	<code>java -version</code>	17.x
2	Tomcat 展開	<code>C:\opt\tomcat\bin\startup.bat</code> 等の存在	直下に <code>bin/conf/lib</code>
3	環境変数	<code>echo %CATALINA_HOME%</code> (新規 cmd)	<code>C:\opt\tomcat</code>
4	サービス登録	<code>Get-Service Tomcat10</code>	サービスが存在
5	サービス稼働	<code>Get-Service Tomcat10</code>	Status: Running
6	HTTP 応答	<code>http://localhost:8080/</code>	初期ページ (200)
7	自動起動	<code>tomcat10w</code> の Startup type	Automatic

すべて確認できれば、フェーズ 4 は完了です。次はフェーズ 5（Shibboleth IdP 5 の導入・LDAP 認証・emailAddress 形式 NameID の準備）です。

9. フェーズ 5 : Shibboleth IdP 5 (テスト IdP)

目的：Windows 上の Tomcat に Shibboleth IdP 5 を導入し、ApacheDS (LDAP) で認証、mail 属性を **emailAddress 形式の NameID** の元として扱う準備までを行う。将来この IdP は Entra ID に差し替え可能 (学習用テスト IdP)。

WSL 版との違い：install.sh→install.bat、Linux パス→Windows パス、systemctl restart →**Tomcat サービス再起動**。SAML の設計 (LDAP 認証・NameID) は共通。NameID 形式は WSL 版の unspecified から **emailAddress** に変更し、元属性を uid→**mail** にする (顧客 Entra の形式に合わせる)。

前提：フェーズ 4 で Tomcat が 8080 で起動確認済み。フェーズ 3 の LDAP 引き継ぎ値 (ldap://localhost:10389/ou=people,dc=example,dc=com/uid=idp-reader,ou=people,dc=example,dc=com/idp-reader/(uid={user})/NameID の元 mail) を使う。

9.1 IdP 5 の入手と展開

- Shibboleth 公式 (<https://shibboleth.net/downloads/identity-provider/latest5/>) から **IdP 5 の zip** (Windows は zip 推奨。Windows 改行) を入手し C:\lab\installers へ。
 - latest5/ には**その時点の最新版だけ**が置かれる。实在版のファイル名を確認してから取得する (例：ブラウザで上記 URL を開き、shibboleth-identity-provider-5.x.y.zip を確認)。
- 任意の場所 (例 C:\lab) に展開。展開先は導入後は不要。

```
Expand-Archive -Path "C:\lab\installers\shibboleth-identity-provider-5.*.zip" -DestinationPath "C:\lab" -Force
Get-ChildItem C:\lab | Where-Object Name -like "shibboleth-identity-provider-5*"
```

9.2 install.bat による導入

展開したディストリビューションフォルダに入り、bin\install.bat を**管理者コマンドプロンプト**で実行します (対話式)。

```
cd /d C:\lab\shibboleth-identity-provider-5.x.y
bin\install.bat
```

対話 (プロンプト) での入力：

質問	入力値
Installation Directory (idp.home)	C:\opt\shibboleth-idp
Host Name	idp.plm-lab.local
SAML EntityID	https://idp.plm-lab.local/idp/shibboleth
Attribute Scope	plm-lab.local
Keystore Password	changeit
Sealer Password	changeit
- 導入すると、署名・暗号化鍵(b) (idp-signing / idp-encryption / sealer 等) が自動生成され、C:\opt\shibboleth-idp\ に配置、war\idp.war がビルドされる。 - RIPEMD-160 などの INFO ログは無害 (SAML では使わない)。	
install.bat が JAVA を見つけれない場合は、JAVA_HOME=C:\opt\jdk-17 (\$5.6) が設定され、新しいコマンドプロンプトで実行しているか確認する。	

9.3 Tomcat への配置 (context フラグメント)

Tomcat に IdP の war を認識させる **context フラグメント** idp.xml を配置します。

1. フォルダを作成（無ければ）：C:\opt\tomcat\conf\Catalina\localhost\
2. idp.xml を作成（管理者権限。昇格 PowerShell 例）：

```
New-Item -ItemType Directory -Force -Path "C:\opt\tomcat\conf\Catalina\localhost" | Out-Null
Set-Content -Path "C:\opt\tomcat\conf\Catalina\localhost\idp.xml" -Encoding ASCII -Value '<C
ontext docBase="C:/opt/shibboleth-idp/war/idp.war" privileged="true" antiResourceLocking="fa
lse" swallowOutput="true" />'
```

docBase のパス区切りは /（スラッシュ）で記述する（Tomcat の context では Windows で
も / が無難）。

JSTL の追加（必須）：IdP 5.2.3 では、状態ページ /idp/profile/status などが JSP+JSTL で
描画されるため、**JSTL を war に追加しないと ClassNotFoundException: jakarta.servlet.js
p.jstl.core.Config – ServletException になる**（メタデータ /idp/shibboleth は JSTL 不要の
ため返るが、status は失敗する）。次の手順で **API と実装（Glassfish）の 2 jar** を overlay
に置き、build.bat で war を再ビルドする。

```
$lib = "C:\opt\shibboleth-idp\edit-webapp\WEB-INF\lib"
New-Item -ItemType Directory -Force -Path $lib | Out-Null
# JSTL API
Invoke-WebRequest -UseBasicParsing -Uri "https://repo1.maven.org/maven2/jakarta/servlet/jsp/jstl/jak
arta.servlet.jsp.jstl-api/3.0.0/jakarta.servlet.jsp.jstl-api-3.0.0.jar" -OutFile "$lib\jakarta.servl
et.jsp.jstl-api-3.0.0.jar"
# JSTL 実装 (Glassfish) ※ 実装が無いと Config クラスが見つからずエラーになる
Invoke-WebRequest -UseBasicParsing -Uri "https://repo1.maven.org/maven2/org/glassfish/web/jakarta.se
rvlet.jsp.jstl/3.0.1/jakarta.servlet.jsp.jstl-3.0.1.jar" -OutFile "$lib\jakarta.servlet.jsp.jstl-3.
0.1.jar"

cd /d C:\opt\shibboleth-idp\bin
build.bat
```

Rebuilding ...war\idp.war / Overlay from ...edit-webapp / Creating war file ...war\idp.war が出れ
ば成功。オフライン環境では別端末で 2 jar を取得し edit-webapp\WEB-INF\lib\ にコピーしてから buil
d.bat する。

9.4 LDAP 認証の設定 (ldap.properties)

C:\opt\shibboleth-idp\conf\ldap.properties を編集し、フェーズ 3 の値に合わせます（**平文 LDAP • 10
389 • dc=example,dc=com**）。主な項目：

idp.authn.LDAP.authenticator	= bindSearchAuthenticator
idp.authn.LDAP.ldapURL	= ldap://localhost:10389
idp.authn.LDAP.useStartTLS	= false
idp.authn.LDAP.useSSL	= false
idp.authn.LDAP.baseDN	= ou=people,dc=example,dc=com
idp.authn.LDAP.subtreeSearch	= true
idp.authn.LDAP.userFilter	= (uid={user})
idp.authn.LDAP.bindDN	= uid=idp-reader,ou=people,dc=example,dc=com
idp.authn.LDAP.dnFormat	= uid=%s,ou=people,dc=example,dc=com
# 返す属性に mail を含める (NameID の元)	
idp.authn.LDAP.returnAttributes	= mail,uid

- ・ **検索バインドのパスワード**は `credentials\secrets.properties` に集約します（WSL 版と同様、二重定義による WARN を避ける）。`C:\opt\shibboleth-idp\credentials\secrets.properties` の該当行：

```
idp.authn.LDAP.bindDNCredential = idp-reader
```

`ldap.properties` 側に `idp.authn.LDAP.bindDNCredential` の行があれば **コメントアウト**して重複を避ける。

- ・ `useStartTLS=false`／`useSSL=false` のため、`trustCertificates`／`trustStore` 系の行は使われない（有効なら不要・コメントアウト可）。インストール直後は `trustCertificates = %\{idp.home}\credentials\ldap-server.crt` 等が有効になっていることがあり、存在しないファイルを指すため紛らわしい。平文 LDAP では両行をコメントアウトしておく。
- ・ 属性解決（`attribute-resolver`）でも同じ LDAP を参照する場合は `idp.attribute.resolver.LDAP.*` を同様に設定するが、本書は NameID の元 mail を `authn` 側の `returnAttributes` で取得する構成を基本とする（フェーズ 10 で属性解放と NameID 生成を確定）。

9.5 emailAddress 形式 NameID の準備 (saml-nameid.xml)

mail 属性を **emailAddress 形式の NameID** として発行する定義を、`C:\opt\shibboleth-idp\conf\saml-nameid.xml` の `<util:list id="shibboleth.SAML2NameIDGenerators">` の内側に追加します（詳細な解放設定はフェーズ 10 で仕上げる。ここでは生成器の準備まで）。

```
<bean parent="shibboleth.SAML2AttributeSourcedGenerator"
  p:omitQualifiers="true"
  p:format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"
  p:attributeSourceIds="#{ {'mail'} }" />
```

これにより、ログインユーザーの mail（例 `01PLM01@plm-lab.local`）が emailAddress 形式の NameID として発行できる状態になる。SP 側での `REMOTE_USER` へのマッピングはフェーズ 10 で行う。

注意（重複させない）：既存のコメントアウト例を有効化する場合は、それとは別に同じ bean を追記しないこと。`shibboleth.SAML2NameIDGenerators` の中に同一の `SAML2AttributeSourcedGenerator` が 2 つ入っていると意図しない重複になる（1 つだけにする）。SAML1 側（`shibboleth.SAML1NameIDGenerators`）は本構成では不要。

9.6 起動と動作確認

`Restart-Service Tomcat10`

`Start-Sleep -Seconds 20`

ステータス（テキストが返る）

`Invoke-WebRequest http://localhost:8080/idp/profile/status -UseBasicParsing | Select-Object -ExpandProperty Content`

メタデータ（`<EntityDescriptor ...>` が返る）

`(Invoke-WebRequest http://localhost:8080/idp/shibboleth -UseBasicParsing).Content.Substring(0,300)`

- ・ `C:\opt\shibboleth-idp\logs\idp-process.log` に致命的 ERROR が無いこと（`bindDNCredential` の Duplicate WARN が出たら §9.4 の集約を確認）。
- ・ `install.bat` 導入直後に `bin\status.bat` を実行すると、既定で `http://localhost/idp/status`（80 番）を見にいき「Connection refused」になることがあるが、これは想定どおり（本構成は 80 80／のちに 8443）。確認は上記 8080 の URL で行う。

9.7 動作確認チェックリスト（フェーズ5）

#	確認内容	期待結果
1	IdP 導入	C:\opt\shibboleth-idp\ に conf/credentials /metadata/war が存在
2	context 配置	C:\opt\tomcat\conf\Catalina\localhost\idp.xml が存在
2b	JSTL 追加	edit-webapp\WEB-INF\lib に JSTL の API+実装 2 jar があり build.bat 済み
3	Tomcat 起動	Tomcat10 が Running
4	ステータス	http://localhost:8080/idp/profile/status が環境情報を返す
5	メタデータ	http://localhost:8080/idp/shibboleth が <EntityDescriptor> を返す
6	LDAP 設定	ldap.properties が 10389 / dc=example,dc=com / idp-reader、returnAttributes に mail
7	NameID 準備	saml-nameid.xml に emailAddress の SAML2AttributeSourcedGenerator (mail)
8	ログ	idp-process.log に致命的 ERROR・重複 WARN が無い

すべて確認できれば、フェーズ5は完了です。次はフェーズ6（Tomcatを直接HTTPS 8443で公開）です。

10. フェーズ6：Tomcat 直 HTTPS（8443）公開

目的：ブラウザからIdPへ**HTTPS（8443）**で到達できるようにする。WSL版では前段にApacheを置いてTLSを終端したが、本構成では**Tomcat自身にHTTPSコネクタを持たせてApacheを省略**する。証明書はフェーズ2で作成したidp.pfx（TLS/HTTPSサーバ証明書(a)・パスワードchangeit）を使う。

WSL版との違い：Apache HTTPDの導入・リバースプロキシ設定・mirroredネットワーク・Listen 80/443無効化がすべて不要。Tomcatのserver.xmlにSSLコネクタを1つ追加するだけ。

10.1 証明書（idp.pfx）をTomcatに配置

フェーズ2で作成したC:\lab\ca\idp.pfxをTomcatのconfに配置します（管理者PowerShell）。

```
Copy-Item "C:\lab\ca\idp.pfx" "C:\opt\tomcat\conf\idp.pfx" -Force
Get-Item "C:\opt\tomcat\conf\idp.pfx"
```

idp.pfxはidp.crt+idp.key+rootCA.crtを含むPKCS12（SAN=idp.plm-lab.local）。パスワードはchangeit（\$3.1）。

10.2 server.xmlにHTTPS 8443コネクタを追加

C:\opt\tomcat\conf\server.xmlを管理者権限で編集します。既存の8080 HTTPコネクタ（<Connector port="8080" .../>）の**近く**に、次のHTTPS 8443コネクタを追加します（Tomcat 10.1は**SSLHostConfig+Certificate**方式）。

```
<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
  maxThreads="150" SSLEnabled="true" scheme="https" secure="true">
  <SSLHostConfig>
    <Certificate certificateKeystoreFile="conf/idp.pfx"
      certificateKeystorePassword="changeit"
      certificateKeystoreType="PKCS12"
      type="RSA" />
  </SSLHostConfig>
</Connector>
```

- certificateKeystoreFile="conf/idp.pfx" は CATALINA_BASE (=C:\opt\tomcat) からの相対で解決される。絶対パスにする場合は C:/opt/tomcat/conf/idp.pfx (スラッシュ) で書く。
- パスワードに <> を含む場合は XML エスケープが必要 (今回の changeit は不要)。

10.3 8080 を localhost 限定にする (任意・推奨)

ブラウザは 8443 経由で IdP にアクセスするため、平文の 8080 は localhost だけに絞っておくと安全です。既存の 8080 コネクタに address="127.0.0.1" を追加します。

```
<Connector port="8080" protocol="HTTP/1.1"
  address="127.0.0.1"
  connectionTimeout="20000"
  redirectPort="8443" />
```

redirectPort を 8443 にしておくと、機密制約でリダイレクトが必要な場合に HTTPS へ誘導される。必須ではない。

10.4 反映と動作確認

Restart-Service Tomcat10

Start-Sleep -Seconds 20

8443 が待受

netstat -ano | findstr 8443

HTTPS で status (CA 検証込み。鍵マーク相当)

Invoke-WebRequest https://idp.plm-lab.local:8443/idp/profile/status -UseBasicParsing | Select-Object StatusCode

- Invoke-WebRequest が証明書エラーを出さずに 200 を返せば、rootCA 信頼 (§6.5) と SAN (idp.plm-lab.local) が正しく効いています。
- ゲスト Windows のブラウザで https://idp.plm-lab.local:8443/idp/profile/status を開き、**鍵マーク (証明書警告なし)** で環境情報が表示されることを確認。
- 起動に失敗する場合は C:\opt\tomcat\logs\catalina.*.log を確認 (多くは certificateKeystoreFile のパス誤り、certificateKeystorePassword 不一致、certificateKeystoreType の指定漏れ)。

証明書警告が出る場合: rootCA が「信頼されたルート証明機関」に入っているか (§6.5、certlm.msc)、アクセス URL のホスト名が idp.plm-lab.local (SAN と一致) か、hosts 解決 (127.0.0.1) を確認。

10.5 動作確認チェックリスト (フェーズ 6)

#	確認内容	期待結果
1	証明書配置	C:\opt\tomcat\conf\idp.pfx が存在
2	server.xml	8443 の SSLHostConfig+Certificate コネクタを追加
3	サービス起動	Tomcat10 が Running (8443 コネクタ起動)

#	確認内容	期待結果
		失敗が無い)
4	待受	netstat で 8443 が LISTENING
5	HTTPS status	https://idp.plm-lab.local:8443/idp/profile/status が 200 (証明書エラーなし)
6	ブラウザ	鍵マークで status 表示 (警告なし)
7	8080	(任意) address="127.0.0.1" で localhost 限定

すべて確認できれば、フェーズ6は完了です。WSL版で必要だったApache前段が不要になり、IdPがブラウザからHTTPSで到達可能になりました。次はフェーズ7（IISの構築・SPの保護対象サイトと443/TLS）です。

11. フェーズ7：IIS（SPの保護対象サイト・443/TLS）

目的：ゲストWindowsにIISを導入し、保護対象となるPLM相当のサイト（認証後に識別子=REMOTE_USERを表示する確認ページ）を443/HTTPSで用意する。フェーズ2の**sp 証明書(a)** (sp.pfx)を初めて使う。フェーズ8でShibboleth SP（IISネイティブモジュール）を組み込む土台。

WSL版との違い：IISもIdP（Tomcat）も同一のWindows上にあり、sp.plm-lab.local-127.0.0.1で素直に届くため、mirroredネットワークの検討は不要。sp.pfxはWSLの\\wsl\$経由ではなく、フェーズ2で作成したC:\lab\ca\sp.pfxから直接取り込む。

11.1 IISの導入（Windowsの機能）

フェーズ8のShibboleth SPはIISモジュールとして動くため、ISAPI拡張／フィルタを含めて有効化します。確認ページ用に**古典ASP**も入れます。**管理者PowerShell**で：

```
Enable-WindowsOptionalFeature -Online -All -FeatureName `
    IIS-WebServerRole, IIS-WebServer, IIS-CommonHttpFeatures, IIS-StaticContent, `
    IIS-DefaultDocument, IIS-ISAPIExtensions, IIS-ISAPIFilter, IIS-ASP, `
    IIS-RequestFiltering, IIS-WebServerManagementTools, IIS-ManagementConsole
```

GUIの場合：「コントロールパネル→プログラム→Windowsの機能の有効化または無効化→インターネットインフォメーションサービス」で、「World Wide Webサービス→アプリケーション開発機能→**ISAPI拡張機能／ISAPIフィルタ／ASP**」と「Web管理ツール→**IIS管理コンソール**」を有効化。

確認：ブラウザでhttp://localhost/にIISの既定ページが表示される。

11.2 hosts（確認）

sp.plm-lab.localはフェーズ2 §6.2で登録済み。未登録なら追加します。

```
ping -n 1 sp.plm-lab.local          # 127.0.0.1 に解決されること
```

11.3 sp 証明書(a)をWindowsに取り込む

フェーズ2で作成したC:\lab\ca\sp.pfxを、Windowsの証明書ストア（ローカルコンピューター\個人）へ取り込みます。パスワードは§3.1のとおりchangeit。

```
Import-PfxCertificate -FilePath "C:\lab\ca\sp.pfx" `
-CertStoreLocation Cert:\LocalMachine\My `
-Password (ConvertTo-SecureString "changeit" -AsPlainText -Force)
```

rootCA はフェーズ 2 §6.5 で「信頼されたルート証明機関」に登録済みのため、ブラウザは sp 証明書を信頼できます。取り込んだ証明書の Thumbprint/Subject (CN=sp.plm-lab.local) が表示されれば成功。

11.4 既定サイトに 443/HTTPS バインドを追加

```
Import-Module WebAdministration
$cert = Get-ChildItem Cert:\LocalMachine\My |
Where-Object { $_.Subject -like "*CN=sp.plm-lab.local*" } | Select-Object -First 1
New-WebBinding -Name "Default Web Site" -Protocol https -Port 443
New-Item -Path "IIS:\SslBindings\0.0.0.0!443" -Value $cert
```

GUI の場合：IIS マネージャー → 「Default Web Site」 → 右側「バインド」 → 「追加」 → 種類 https/ポート 443/SSL 証明書に sp.plm-lab.local を選択。

11.5 確認用ページ (REMOTE_USER 表示)

C:\inetpub\wwwroot\whoami.asp を作成します。昇格したプロセス（管理者 PowerShell）で作るのが確実です。

```
$asp = @"
<%@ Language="VBScript" %>
<%
Response.CodePage = 65001
Response.Charset = "utf-8"
%>
<html><body>
<h2>Authentication Test</h2>
REMOTE_USER = [<%= Request.ServerVariables("REMOTE_USER") %>]<br>
AUTH_TYPE = [<%= Request.ServerVariables("AUTH_TYPE") %>]
</body></html>
"@
Set-Content -Path "C:\inetpub\wwwroot\whoami.asp" -Value $asp -Encoding UTF8
```

この段階では SP 未導入のため REMOTE_USER は空 ([]) で表示されます。これは正常で、フェーズ 8~10 で SAML 認証が通ると、ここに識別子（メール形式 01PLM01@plm-lab.local）が入ります。

補足 1（ファイル作成権限・OS 差異）：C:\inetpub\wwwroot への書き込みは昇格したプロセスで行う。ビルトイン Administrator の挙動は OS で異なり、Windows Server 2016 は既定で管理者承認モードが無効のためエクスプローラーの「新規作成」で wwwroot に直接書けるが、Windows 11 クライアントは承認モードが効き、エクスプローラーが標準トークンで動くため書けない（UAC スライダーを最下段にしても変わらない）。承認モードの無効化はセキュリティ低下のため非推奨で、昇格プロセス（管理者 PowerShell/「管理者として実行」したメモ帳）での作成を推奨。組織の検証環境（Server 系）ではこの制限自体が起きにくい。

補足 2（文字コード）：日本語を含む古典 ASP を英語ロケールの IIS で表示すると文字化けすることがある。上記のように先頭で Response.CodePage=65001/Response.Charset="utf-8" を指定し、BOM 無し UTF-8 で保存すれば回避できる。確認用途では見出しを英語表記（例：Authentication Test）にしておくのが無難。

11.6 動作確認

ゲスト Windows のブラウザで確認します。

#	確認内容	期待結果
1	IIS 既定ページ	http://localhost/ が表示される
2	HTTPS バインド	https://sp.plm-lab.local/ が鍵マーク（証明書エラーなし）で表示
3	ASP 動作	https://sp.plm-lab.local/whoami.asp が表示される
4	REMOTE_USER	同ページで REMOTE_USER = []（空。SP 未導入のため正常）

証明書警告が出る場合は rootCA が「信頼されたルート証明機関」にあるか（\$6.5）を確認。
whoami.asp が 500 等になる場合は、古典 ASP（\$11.1 の IIS-ASP）が有効か、拡張子が .asp かを確認。

11.7 フェーズ 8 への引き継ぎ値

項目	値
SP entityID	https://sp.plm-lab.local/shibboleth
保護対象（予定）	サイト全体（フェーズ 8 で SP が保護し、REMOTE_USER に識別子が入る）
SP 方式	IIS ネイティブモジュール（ISAPI 有効化済み）
サイト	Default Web Site（443/HTTPS、sp 証明書 (a)）

すべて確認できれば、フェーズ 7 は完了です。次はフェーズ 8（Shibboleth SP を IIS に組み込み、サイト全体を保護）です。

12. フェーズ 8：Shibboleth SP（IIS ネイティブモジュール・サイト全体保護）

目的：ゲスト Windows に Shibboleth SP 3 を導入し、IIS と連携させて Default Web Site 全体を SAML 保護する。SP の SAML 署名・暗号鍵 (b) を生成し、SP メタデータを取得できる状態にする。実際の SSO 成立はフェーズ 9（メタデータ相互登録）以降で完成する。

前提：フェーズ 7 で IIS（ISAPI 有効）・443/HTTPS・確認ページ whoami.asp を用意済み。SP は shibd デーモンと IIS ネイティブモジュールの 2 つで構成される。

WSL 版との違い：SP は元々 Windows ネイティブなので、この部分は WSL 版とほぼ同一。IdP メタデータの :8443 補正はフェーズ 9 で不要になる（本構成の IdP は最初から 8443 直公開のため。詳細はフェーズ 9）。REMOTE_USER に載せる識別子は、フェーズ 10 で **emailAddress 形式 NameID** を割り当てる。

12.1 SP インストーラの入手と導入

- 公式サイト <https://shibboleth.net/downloads/service-provider/latest/> の **win64/** から最新の **msi** を入手（バージョンは固定せず最新を使用）。C:\lab\installers に保存。

- MSI を実行し、既定のまま進める。要点：
 - インストール先は既定の C:\opt\shibboleth-sp。
 - 「Configure IIS support」に**チェック**（IIS ネイティブモジュールを自動構成。実機の MSI ではこの表記）。
 - 完了後、**再起動**を求められるので再起動する。

配置の注記（設計上の推奨）：インストール先は**デフォルトの C:\opt\shibboleth-sp を推奨**。C:\inetpub 配下は、SP の秘密鍵・設定が Web 公開領域に入り漏洩リスクがあるため**不可**。集約する場合も空白を含まないパスにとどめ、shibboleth2.xml 内のパス・keygen 出力先・ログ/鍵のパスを一斉に読み替える必要がある。学習・検証ではデフォルトが最も安全。

12.2 インストールの確認

- サービス：「サービス」管理コンソールで **Shibboleth Daemon (Default)** が「実行中／自動／Local System」であること（内部のサービス名は shibd_Default）。
- IIS（SP3 のネイティブモジュール方式）：IIS マネージャーでサーバーを選択 → 「モジュール」に **ShibNative／ShibNative32**（C:\opt\shibboleth-sp\lib64\shibboleth\iis7_shib.dll 等、Native/Local）があること。※ SP3 はネイティブモジュール方式のため、旧 ISAPI 方式の *.sso ハンドラーマッピングは**表示されないのが正常**。下のステータス確認が通れば実質確認済み。
- ステータス（**必ず localhost で、大文字小文字を区別**）：ブラウザで https://localhost/Shibboleth.sso/Status を開き、末尾に <Status><OK/></Status> が返ること。

動かない場合は shibd の設定チェック：管理者コマンドプロンプトで shibd.exe -check を実行（overall configuration is loadable... なら設定は読み込み可能）。**shibd.exe のパスは環境により sbin64 または sbin のいずれか**：- 64bit : C:\opt\shibboleth-sp\sbin64\shibd.exe -check - 32bit : C:\opt\shibboleth-sp\sbin\shibd.exe -check

どちらにあるかは dir C:\opt\shibboleth-sp\sbin64\shibd.exe / dir C:\opt\shibboleth-sp\sbin\shibd.exe で確認できる。ログは C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log。

12.3 shibboleth2.xml の編集

C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml を編集します（**まず shibboleth2.xml.orig にバックアップ**。タイプミスが最大の事故要因なので慎重に。特に sp.example.org の置換漏れに注意）。変更点は次の 4 か所です。

(1) <ISAPI> の <Site>（IIS サイト ID とホスト名の対応。Default Web Site の ID は通常 1）

```
<ISAPI normalizeRequest="true" safeHeaderNames="true">
  <Site id="1" name="sp.plm-lab.local" scheme="https" port="443"/>
</ISAPI>
```

(2) <RequestMapper> の <Host>（**サイト全体を保護**）

```
<RequestMapper type="Native">
  <RequestMap>
    <Host name="sp.plm-lab.local" authType="shibboleth" requireSession="true"/>
  </RequestMap>
</RequestMapper>
```

requireSession="true" により、このホストへの**全アクセスがセッション必須（＝未認証なら IdP へ）**になります。これが「**サイト全体保護**」の設定です。

(3) <ApplicationDefaults> の entityID と REMOTE_USER

```
<ApplicationDefaults entityID="https://sp.plm-lab.local/shibboleth"
    REMOTE_USER="eppn persistent-id targeted-id">
```

REMOTE_USER は「先頭から最初に値のある属性」が採用されます。**識別子 (emailAddress 形式 NameID) をここに載せる最終設定はフェーズ 10**で行います (IdP 側の NameID/属性の出し方と対で決めるため)。本フェーズでは既定のままにしておきます。

(4) <SSO> に IdP の entityID を指定

```
<SSO entityID="https://idp.plm-lab.local/idp/shibboleth">
    SAML2
</SSO>
```

IdP のメタデータ (<MetadataProvider>) の登録は**フェーズ 9**で行います。本フェーズでは entityID の指定までにとどめます。<MetadataProvider>を追加する際は、<Sessions>の外・<Errors>の後ろに置く (<Sessions>内に置くとスキーマ違反で shibd が起動しない。フェーズ 9 で詳述)。

12.4 SP 鍵 (b) の生成

SP の SAML 署名・暗号化証明書 (b) を、正しいホスト名・entityID で生成します。管理者コマンドプロンプトで：

```
cd C:\opt\shibboleth-sp\etc\shibboleth
keygen.bat -h sp.plm-lab.local -e https://sp.plm-lab.local/shibboleth -y 10
```

sp-signing-cert.pem / sp-encrypt-cert.pem 等が生成されます。これはフェーズ 2 の (a) とは別物で、**メタデータ交換で IdP に渡す (b)** (CA 信頼・ホスト名一致は不要)。MSI が既定鍵を生成済みの場合もありますが、entityID/ホスト名を正しくするため上記で作り直します。

12.5 反映 (IIS 完全再起動)

<ISAPI> を変更したときは **IIS の完全再起動**が必要です。

```
Restart-Service shibd_Default
iisreset
```

12.6 動作確認

#	確認内容	期待結果
1	shibd 稼働	「Shibboleth Daemon (Default)」が実行中
2	設定読込	shibd.exe -check (sbin64 または sbin) が overall configuration is loadable
3	Status	https://localhost/Shibboleth.sso/Status が <Status><OK></Status>
4	SP メタデータ	https://sp.plm-lab.local/Shibboleth.sso/Metadata が SP メタデータ (XML) を返す。中に sp.example.org が残っていない (すべて sp.plm-lab.local)
5	保護の発火	https://sp.plm-lab.local/whoami.asp にアクセスすると SP がセッションを要求する

#	確認内容	期待結果
		(この時点では IdP メタデータ未登録のため No MetadataProvider available. 等になるのは 想定どおり 。requireSession が効いている証拠)

確認 4 の SP メタデータ (Shibboleth.sso/Metadata) は、フェーズ 9 で **IdP に登録する SP メタデータ** として使います。ファイルに保存しておくと便利です。

12.7 フェーズ 9 への引き継ぎ値

項目	値
SP entityID	https://sp.plm-lab.local/shibboleth
SP メタデータ URL	https://sp.plm-lab.local/Shibboleth.sso/Metadata
SP の ACS (想定)	https://sp.plm-lab.local/Shibboleth.sso/SAML2/POST
SP 署名・暗号鍵 (b)	C:\opt\shibboleth-sp\etc\shibboleth\sp-*-cert.pem
次工程	フェーズ 9: IdP メタデータを SP に登録し、SP メタデータを IdP に登録。 本構成では IdP が 8443 直公開のため: 8443 補正は不要

すべて確認できれば、フェーズ 8 は完了です。次はフェーズ 9 (メタデータ交換・初回 SSO 成立) です。

13. フェーズ 9: メタデータ交換 (IdP ↔ SP の相互信頼・初回 SSO 成立)

目的: IdP と SP のメタデータを静的 (ファイル) に相互登録し、両者が信頼し合って SAML の往復が成立する状態にする。フェーズ 8 で出た No MetadataProvider available. を解消し、https://sp.plm-lab.local/whoami.asp への未認証アクセスが **IdP のログイン画面 (8443) へ遷移 → 識別子+パスワードでログイン → SP に戻ってセッション確立**、という一連を確認する。

本フェーズのゴール: 「SSO の往復が成立し、SP セッションが張れて保護ページに到達できる」ところまで。REMOTE_USER に識別子を載せる最終設定はフェーズ 10。本フェーズ完了時点では REMOTE_USER は空でも合格。

WSL 版との違い (:8443 補正・実機結果): \\wsl\$・/mnt/c の受け渡しや chown は不要で、すべて同一 Windows 上のファイルコピーで済む。ただし **install.bat はホスト名ベースでメタデータを生成するため、エンドポイントの Location がポートなし (=443) で出力されることがある** (実機でもポートなしだった)。その場合は WSL 版と同様に **:8443 へ補正が必要** (補正の手段が sed+PowerShell に変わるだけ)。13.1 で実際の Location を確認し、ポートなしなら補正する。

13.1 IdP メタデータのエンドポイント確認と :8443 補正

IdP のメタデータは C:\opt\shibboleth-idp\metadata\idp-metadata.xml にあります。まず、各エンドポイントの Location が **:8443 付きかポートなし (=443) か**を確認します (管理者 PowerShell)。

```
Select-String -Path "C:\opt\shibboleth-idp\metadata\idp-metadata.xml" -Pattern 'Location="[^"]*"' |
  ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
```

実機ではポートなし (https://idp.plm-lab.local/idp/...) で生成された。このままだと SP はブラウザを 443 (IIS/SP 側) へ送ってしまい SSO が壊れるため、SP 側へコピーしてから **:8443 へ補正**する (entityID 行はポートなしのまま戻す)。

1) IdP メタデータを SP 側へコピー

```
Copy-Item "C:\opt\shibboleth-idp\metadata\idp-metadata.xml" `
  "C:\opt\shibboleth-sp\etc\shibboleth\idp-metadata.xml" -Force
```

2) コピー先のエンドポイントを :8443 に補正 (entityID は元に戻す)

```
$f = "C:\opt\shibboleth-sp\etc\shibboleth\idp-metadata.xml"
(Get-Content $f -Raw) `
  -replace 'https://idp.plm-lab.local/idp/', 'https://idp.plm-lab.local:8443/idp/' `
  -replace 'entityID="https://idp.plm-lab.local:8443/idp/shibboleth"', 'entityID="https://idp.plm-lab.local/idp/shibboleth"' |
  Set-Content $f -Encoding UTF8
```

3) 確認: Location が :8443、entityID はポートなし

```
Select-String -Path $f -Pattern 'Location="[^"]*"' | ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
Select-String -Path $f -Pattern 'entityID="[^"]*"' | ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
```

期待: Location がすべて https://idp.plm-lab.local:8443/idp/...、entityID は https://idp.plm-lab.local/idp/shibboleth (**ポートなし**。識別子であってアクセス先 URL ではないため、ポートを付けない)。もし最初から Location が :8443 付きで生成されていた場合は、手順 1 のコピーのみで補正は不要。

13.2 SP に IdP メタデータを登録

C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml に IdP メタデータの <MetadataProvider> を追加します。**配置場所が重要で、<MetadataProvider> は <Sessions> の中ではなく、</Sessions> の後・<Errors .../> の下に置きます** (<Sessions> 内に置くとスキーマ違反 element 'MetadataProvider' is not allowed for content model ... で shibd が起動しません)。既定ファイルの「Example of locally maintained metadata」コメントの位置がまさにその場所です。

```
</Sessions>

<Errors supportContact="root@localhost"
  helpLocation="/about.html"
  styleSheet="/shibboleth-sp/main.css"/>

<!-- ここ (Sessions の外・Errors の下) に追加 -->
<MetadataProvider type="XML" validate="true" path="idp-metadata.xml"/>
```

保存後、設定チェックしてから SP を再起動します (shibd.exe は sbin64 または sbin)。

```
C:\opt\shibboleth-sp\sbin64\shibd.exe -check # 無ければ sbin\shibd.exe -check
Restart-Service shibd_Default
iisreset
```

これで No MetadataProvider available. は解消します。https://localhost/Shibboleth.sso/Status が引き続き <OK/> であることも確認。

13.3 SP メタデータの取得と IdP への配置

SP のメタデータを取得して IdP 側へ配置します。ブラウザで <https://sp.plm-lab.local/Shibboleth.sso/Metadata> を開き、**XML を sp-metadata.xml として保存**（フェーズ 8 で保存済みならそれを使用）。これを IdP のメタデータ領域へコピーします（同一 Windows なので単純コピー）。

```
Copy-Item "C:\Users\<ユーザー>\Downloads\sp-metadata.xml" `
"C:\opt\shibboleth-idp\metadata\sp-metadata.xml" -Force
```

WSL 版のような chown tomcat は不要（Windows のサービスは Local System で動作し、ファイル所有権の付け替えは不要）。

13.4 IdP に SP メタデータを登録

C:\opt\shibboleth-idp\conf\metadata-providers.xml を編集し、既定の `<MetadataProvider id="ShibbolethMetadata" xsi:type="ChainingMetadataProvider">` と、それを閉じる `</MetadataProvider>` の間（**チェーンの内側**）に、SP メタデータの `FilesystemMetadataProvider` を追加します（コメントアウトされた Local Metadata 見本の位置が最適）。

```
<MetadataProvider id="LocalSP" xsi:type="FilesystemMetadataProvider"
    metadataFile="%{idp.home}/metadata/sp-metadata.xml"/>
```

IdP に設定を反映させます。学習環境では **Tomcat 再起動が確実** です（reload-service はアクセス制御で弾かれることがある）。

```
Restart-Service Tomcat10
Start-Sleep -Seconds 20
# SP メタデータ (entityID: https://sp.plm-lab.local/shibboleth) の読み込み・エラー無しを確認
Select-String -Path "C:\opt\shibboleth-idp\logs\idp-process.log" -Pattern 'LocalSP|sp.plm-lab.local'
| Select-Object -Last 10
# 期待: "FilesystemMetadataResolver LocalSP: New metadata successfully loaded ..." が出る
```

13.5 時刻の確認

SAML はクロックスキューに敏感ですが、**本構成は IdP も SP も同一の Windows 上**のため時刻は常に一致し、WSL 版のような OS 間のずれは発生しません。Get-Date が妥当な現在時刻であることを確認する程度で十分です。

13.6 初回 SSO の確認

ゲスト Windows のブラウザ（新しいプライベートウィンドウ推奨）で：

1. <https://sp.plm-lab.local/whoami.asp> を開く
2. **IdP のログイン画面**（<https://idp.plm-lab.local:8443/idp/...>）に遷移する
3. ユーザー名 01PLM01、パスワード 01PLM01（フェーズ 3 の Joe アカウント）でログイン
4. **SP に戻り、whoami.asp が表示される**（保護ページに到達）

#	確認内容	期待結果
1	保護の発火+遷移	未認証アクセスで IdP ログイン画面（8443）へ遷移
2	認証	uid=01PLM01 でログインできる（LDAP 認証成立）
3	復路	SP に戻り、whoami.asp が開ける（セッ

#	確認内容	期待結果
4	セッション	セッション確立) https://sp.plm-lab.local/Shibboleth.sso/Session に有効なセッションが見える (localhost では不可 。ログインしたホスト名で開く)

この時点で REMOTE_USER は空でも合格。識別子 (emailAddress 形式 NameID) を REMOTE_USER に載せるのはフェーズ 10。

13.7 つまづいたときの切り分け

- **IdP ログイン画面に飛ばず 443 に行ってしまう** → 13.1 のエンドポイントが :8443 になっているか。SP 側 idp-metadata.xml の Location を確認。
- **shibd が起動しない** / element 'MetadataProvider' is not allowed ... → <MetadataProvider> を <Sessions> 内に置いている。</Sessions> の後・<Errors> の下へ移動 (13.2)。
- **IdP 側で「SAML2 SSO profile is not configured for relying party ...sp...」** → IdP に SP メタデータが読めていない (13.4)。idp-process.log と metadata-providers.xml のパス・SP メタデータの entityID を確認。
- **署名/復号エラー** → メタデータ内の (b) 証明書と実鍵の不一致。SP/IdP のメタデータが最新か (keygen 後に再取得したか) を確認。
- ログ: SP は C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log、IdP は C:\opt\shibboleth-idp\logs\idp-process.log。

すべて確認できれば、フェーズ 9 は完了です (初回 SSO 成立)。次はフェーズ 10 (emailAddress 形式 NameID を REMOTE_USER に載せる) です。

14. フェーズ 10：属性連携 (emailAddress 形式 NameID を REMOTE_USER に載せる)

目的: 認証されたユーザーの mail (例 01PLM01@plm-lab.local) を、IdP から **emailAddress 形式の NameID** として SP へ渡し、SP 側で REMOTE_USER にマッピングする。最終的に whoami.asp の REMOTE_USER に 01PLM01@plm-lab.local が表示される状態にする (顧客 Entra の形式に準拠)。

WSL 版との違い: WSL 版は「uid を unspecified 形式の NameID」だったが、本構成は「**mail を emailAddress 形式の NameID**」。顧客の本番 (Entra ID) が emailAddress 形式の NameID を送る構成に合わせている。PLM 側で @ の前を切り出して従来の識別番号に変換する処理は **PLM アプリの責務**であり、本フェーズの対象外 (SP は 01PLM01@plm-lab.local をそのまま REMOTE_USER に載せるところまで)。設定は最も細かいので、**1 か所ずつ変更して確認**する。

14.1 IdP：mail 属性の解決 (attribute-resolver.xml)

mail を LDAP から取得できるようにします。IdP 5 の attribute-resolver.xml に LDAP DataConnector と mail の AttributeDefinition が無い場合は追加します (ldap.properties の接続情報を使う既定の %{idp.attribute.resolver.LDAP.*} を参照)。

```
<!-- LDAP から属性を引く DataConnector (既定で例がコメントアウトされていることが多い) -->
<DataConnector id="myLDAP" xsi:type="LDAPDirectory"
  ldapURL="%{idp.attribute.resolver.LDAP.ldapURL}"
  baseDN="%{idp.attribute.resolver.LDAP.baseDN}"
```

```

principal="%{idp.attribute.resolver.LDAP.bindDN}"
principalCredential="%{idp.attribute.resolver.LDAP.bindDNCredential}">
<FilterTemplate><![CDATA[(uid=$resolutionContext.principal)]]></FilterTemplate>
<ReturnAttributes>mail uid</ReturnAttributes>
</DataConnector>

<!-- mail 属性の定義 (DataConnector から取得) -->
<AttributeDefinition id="mail" xsi:type="Simple">
  <InputDataConnector ref="myLDAP" attributeNames="mail"/>
</AttributeDefinition>

```

attribute-resolver.xml に対応する idp.attribute.resolver.LDAP.* を ldap.properties に設定しておく（フェーズ 5 で authn 用に設定した値と同じ：ldapURL=ldap://localhost:10389/ba seDN=ou=people,dc=example,dc=com/bindDN=uid=idp-reader,ou=people,dc=example,dc=com、bind DNCredential は secrets.properties）。既に mail が status の属性解決で取得できていれば、この節は最小限で済む。

14.2 IdP：mail を対象 SP へ解放 (attribute-filter.xml)

C:\opt\shibboleth-idp\conf\attribute-filter.xml に、SP (sp.plm-lab.local) へ mail を解放するポリシーを追加します。

```

<AttributeFilterPolicy id="releaseMailToPlmSP">
  <PolicyRequirementRule xsi:type="Requester"
    value="https://sp.plm-lab.local/shibboleth"/>
  <AttributeRule attributeID="mail">
    <PermitValueRule xsi:type="ANY"/>
  </AttributeRule>
</AttributeFilterPolicy>

```

14.3 IdP：mail を emailAddress 形式 NameID として生成 (saml-nameid.xml)

フェーズ 5 §9.5 で、shibboleth.SAML2NameIDGenerators に次の生成器を追加済みのはずです（未追加ならここで追加。重複させない）。

```

<bean parent="shibboleth.SAML2AttributeSourcedGenerator"
  p:omitQualifiers="true"
  p:format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"
  p:attributeSourceIds="#{ {'mail'} }" />

```

14.4 IdP：対象 SP に emailAddress 形式を優先させる (relying-party.xml)

C:\opt\shibboleth-idp\conf\relying-party.xml の <util:list id="shibboleth.RelyingPartyOverrides"> に、対象 SP 向けのオーバーライドを追加します。

```

<bean parent="RelyingPartyByName"
  c:relyingPartyIds="https://sp.plm-lab.local/shibboleth">
  <property name="profileConfigurations">
    <list>
      <bean parent="SAML2.SSO"
        p:nameIDFormatPrecedence="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"/>
    </list>
  </property>
</bean>

```

設定を反映（Tomcat 再起動が確実）。

```
Restart-Service Tomcat10
Start-Sleep -Seconds 20
Select-String -Path "C:\opt\shibboleth-idp\logs\idp-process.log" -Pattern 'ERROR' | Select-Object -Last 20
```

14.5 SP : NameID を REMOTE_USER にマップ

(1) **attribute-map.xml** (C:\opt\shibboleth-sp\etc\shibboleth\attribute-map.xml) に、emailAddress 形式の NameID を属性 mail として取り込むデコーダを追加します。

```
<Attribute name="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress" id="mail">
  <AttributeDecoder xsi:type="NameIDAttributeDecoder" formatter="$Name"/>
</Attribute>
```

(2) **shibboleth2.xml** の <ApplicationDefaults> の REMOTE_USER に、先頭で mail を使うよう変更します（顧客の優先リスト方式に倣う）。

```
<ApplicationDefaults entityID="https://sp.plm-lab.local/shibboleth"
  REMOTE_USER="mail eppn persistent-id targeted-id">
```

反映（SP 再起動。shibd.exe は sbin64 または sbin）。

```
C:\opt\shibboleth-sp\sbin64\shibd.exe -check # overall configuration is loadable
Restart-Service shibd_Default
iisreset
```

14.6 動作確認（最終ゴール）

ゲスト Windows のブラウザの新しいプライベートウィンドウで：

1. <https://sp.plm-lab.local/whoami.asp> を開く
2. IdP ログイン画面（8443）→ 01PLM01 / 01PLM01 でログイン
3. whoami.asp に戻り、REMOTE_USER = [01PLM01@plm-lab.local] と表示される

#	確認内容	期待結果
1	SSO 往復	ログイン後 whoami.asp に戻れる
2	識別子の受け渡し	REMOTE_USER = [01PLM01@plm-lab.local] （メール形式が入る）
3	セッション内容	https://sp.plm-lab.local/Shibboleth.sso/Session の Attributes に mail（NameID）が見える

これが表示できれば、本手順書の目標（emailAddress 形式の識別子による SSO 連携）は達成です。

14.7 つまずいたときの切り分け

- **REMOTE_USER が空のまま** → ① IdP が NameID を出しているか（Shibboleth.sso/Session の Attributes/NameID を確認）、② SP の attribute-map.xml の形式（emailAddress）と id（mail）が一致しているか、③ REMOTE_USER の先頭に mail があるか。
- **NameID が transient のまま** → 14.4 の nameIDFormatPrecedence（emailAddress）が対象 SP に効いているか。relying-party.xml の entityID を確認。
- **mail が解決できない（IdP エラー）** → 14.1 の LDAP DataConnector と ldap.properties の resolver 設定、LDAP の mail 属性（フェーズ 3 で投入済み）を確認。idp-process.log を確認。

- ・ **確認ツール**：ブラウザ拡張「SAML-tracer」で、IdP-SP の SAML Response 内の <NameID> に 01PLM01@plm-lab.local が入っているかを直接確認できる。
- ・ ログ：SP=C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log、IdP=C:\opt\shibboleth-idp\logs\idp-process.log。

14.8 実運用（PLM）への接続に向けた補足

本フェーズで REMOTE_USER にメール形式の識別子（01PLM01@plm-lab.local）が入るようになった。実際の PLM では、この REMOTE_USER（IIS のサーバ変数）を PLM 側が読み取り、**@の前（01PLM01）を切り出して従来の識別番号として自 DB で認可判定する**。この切り出し・変換は PLM アプリ（Web.config の認証方式スイッチ+実装）の責務であり、SP/IdP の構築範囲外。SP はサイト全体を保護しているため、PLM の各ページはすべて認証必須となる。顧客本番では IdP が Entra ID に替わるが、SP 側（attribute-map/REMOTE_USER）の考え方は同じ。

以降のフェーズ（\$15 フェーズ 11、および付録）は、フェーズごとに実機検証しながら順次追記します。